# Real Coder (mattock_name) Guidelines

# 📂Changelog

| Date | Description |
|---|---|
| Mar 18, 2026 | **Added Mandatory Determinism Rule (Step 1a).** Prohibits the use of non-deterministic language such as "or," "recommended," "should," and "etc." to ensure the prompt acts as a strict state machine with a single execution path. |
| Mar 18, 2026 | **Updated Step 1b: Requirement Philosophy.** Established a clear hierarchy for "Overly Specific" (Critical Fail) vs. "Nice to Have" (Rubric Only) to prevent task overfitting and ensure implementation-agnostic verification. |
| Mar 18, 2026 | **Introduced 🧪 STEP 1c: Checking Your Re-written prompt.** Added mandatory auditing protocols for Linguistic Determinism and Structural Compliance, incorporating the promptchecker.md resource for Expected Interface validation. |
| Mar 13, 2026 | Any type of coding agent is ok to be used in this project! See Coding Agent Allowed.<br>We recommend OpenCode over the use of Cursor due to cost effectiveness (**https://opencode.ai/**). Link it to your favorite LLM account to get started! |
| Mar 11, 2026 | When building your website, ensure the UI meets professional freelance standards and all user flows are fully functional.<br>Even if UI design is not a strong requirement if the seed prompt didn't mention it, please make sure the website at least looks good & acceptable for freelance work standards! |
| Mar 10, 2026 | **Visual Asset Compliance**<br>• **Commercially Free Assets:** If your task involves using any visuals including **icons, stock images, illustrations, or fonts**, you must ensure they are **100% commercially free to use**, and **100% ok to use their content for AI and ML development.**<br>• **License Verification:** Do not include assets that require a paid license, attribution (unless specified), or have restrictive copyright tags.<br>• **Example Sources:** Use resources like **Google Fonts, Lucide/Heroicons, or Pexels** (where applicable) to ensure the "Golden Patch" is legally deployable.<br>• If you are not sure, please only use an image placeholder in your code!<br>**!!Do NOT USE any content from Unsplash! !!** |
| Mar 3, 2026 | 🛑If you have a **frontend development / Fullstack development** task, make sure to use Instruction Following (IF) rubrics to cover any UI Design |

| Date | Description |
|---|---|
| Mar 18, 2026 | **Added Mandatory Determinism Rule (Step 1a).** Prohibits the use of non-deterministic language such as "or," "recommended," "should," and "etc." to ensure the prompt acts as a strict state machine with a single execution path. |
| Mar 18, 2026 | **Updated Step 1b: Requirement Philosophy.** Established a clear hierarchy for "Overly Specific" (Critical Fail) vs. "Nice to Have" (Rubric Only) to prevent task overfitting and ensure implementation-agnostic verification. |
| Mar 18, 2026 | **Introduced 🧪 STEP 1c: Checking Your Re-written prompt.** Added mandatory auditing protocols for Linguistic Determinism and Structural Compliance, incorporating the promptchecker.md resource for Expected Interface validation. |
| Mar 13, 2026 | Any type of coding agent is ok to be used in this project! See Coding Agent Allowed.<br>We recommend OpenCode over the use of Cursor due to cost effectiveness (**https://opencode.ai/**). Link it to your favorite LLM account to get started! |
| Mar 11, 2026 | When building your website, ensure the UI meets professional freelance standards and all user flows are fully functional.<br>Even if UI design is not a strong requirement if the seed prompt didn't mention it, please make sure the website at least looks good & acceptable for freelance work standards! |
| Mar 10, 2026 | **Visual Asset Compliance**<br>• **Commercially Free Assets:** If your task involves using any visuals including **icons, stock images, illustrations, or fonts**, you must ensure they are **100% commercially free to use**, and **100% ok to use their content for AI and ML development.**<br>• **License Verification:** Do not include assets that require a paid license, attribution (unless specified), or have restrictive copyright tags.<br>• **Example Sources:** Use resources like **Google Fonts, Lucide/Heroicons, or Pexels** (where applicable) to ensure the "Golden Patch" is legally deployable.<br>• If you are not sure, please only use an image placeholder in your code!<br>**!!Do NOT USE any content from Unsplash! !!** |
| | requirements in your re-written prompt!<br>🛑If you do NOT want to cover so many UI Design requirements, DO NOT include design asks in your re-written prompt! |

| Date | Description |
| --- | --- |
| Mar 18, 2026 | **Added Mandatory Determinism Rule (Step 1a).** Prohibits the use of non-deterministic language such as "or," "recommended," "should," and "etc." to ensure the prompt acts as a strict state machine with a single execution path. |
| Mar 18, 2026 | **Updated Step 1b: Requirement Philosophy.** Established a clear hierarchy for "Overly Specific" (Critical Fail) vs. "Nice to Have" (Rubric Only) to prevent task overfitting and ensure implementation-agnostic verification. |
| Mar 18, 2026 | **Introduced 🧪 STEP 1c: Checking Your Re-written prompt.** Added mandatory auditing protocols for Linguistic Determinism and Structural Compliance, incorporating the promptchecker.md resource for Expected Interface validation. |
| Mar 13, 2026 | Any type of coding agent is ok to be used in this project! See Coding Agent Allowed.<br>We recommend OpenCode over the use of Cursor due to cost effectiveness (**https://opencode.ai/**). Link it to your favorite LLM account to get started! |
| Mar 11, 2026 | When building your website, ensure the UI meets professional freelance standards and all user flows are fully functional.<br>Even if UI design is not a strong requirement if the seed prompt didn't mention it, please make sure the website at least looks good & acceptable for freelance work standards! |
| Mar 10, 2026 | **Visual Asset Compliance**<br>• **Commercially Free Assets:** If your task involves using any visuals including **icons, stock images, illustrations, or fonts**, you must ensure they are **100% commercially free to use**, and **100% ok to use their content for AI and ML development.**<br>• **License Verification:** Do not include assets that require a paid license, attribution (unless specified), or have restrictive copyright tags.<br>• **Example Sources:** Use resources like **Google Fonts, Lucide/Heroicons, or Pexels** (where applicable) to ensure the "Golden Patch" is legally deployable.<br>• If you are not sure, please only use an image placeholder in your code!<br>**!!Do NOT USE any content from Unsplash! !!** |
| Mar 1, 2026 | Your Golden Patch should be answering every single requirement from your re-written prompt, and passing every rubric criteria!<br>If you found that your Golden Patch is missing any rubric criteria, go back and debug your Golden Patch until it is meeting all rubric criterias & passing all the |

| Date | Description |
| --- | --- |
| Mar 18, 2026 | **Added Mandatory Determinism Rule (Step 1a).** Prohibits the use of non-deterministic language such as "or," "recommended," "should," and "etc." to ensure the prompt acts as a strict state machine with a single execution path. |
| Mar 18, 2026 | **Updated Step 1b: Requirement Philosophy.** Established a clear hierarchy for "Overly Specific" (Critical Fail) vs. "Nice to Have" (Rubric Only) to prevent task overfitting and ensure implementation-agnostic verification. |
| Mar 18, 2026 | **Introduced 🧪 STEP 1c: Checking Your Re-written prompt.** Added mandatory auditing protocols for Linguistic Determinism and Structural Compliance, incorporating the promptchecker.md resource for Expected Interface validation. |
| Mar 13, 2026 | Any type of coding agent is ok to be used in this project! See Coding Agent Allowed.<br>We recommend OpenCode over the use of Cursor due to cost effectiveness (**https://opencode.ai/**). Link it to your favorite LLM account to get started! |
| Mar 11, 2026 | When building your website, ensure the UI meets professional freelance standards and all user flows are fully functional.<br>Even if UI design is not a strong requirement if the seed prompt didn't mention it, please make sure the website at least looks good & acceptable for freelance work standards! |
| Mar 10, 2026 | **Visual Asset Compliance**<br>• **Commercially Free Assets:** If your task involves using any visuals including **icons, stock images, illustrations, or fonts**, you must ensure they are **100% commercially free to use**, and **100% ok to use their content for AI and ML development.**<br>• **License Verification:** Do not include assets that require a paid license, attribution (unless specified), or have restrictive copyright tags.<br>• **Example Sources:** Use resources like **Google Fonts, Lucide/Heroicons, or Pexels** (where applicable) to ensure the "Golden Patch" is legally deployable.<br>• If you are not sure, please only use an image placeholder in your code!<br>**!!Do NOT USE any content from Unsplash! !!** |
| | tests! |
| Feb 27, 2026 | Added **Another System Prompt Folder!** |
| Feb 27, 2026 | Added System Prompt Package for You to Use as Off-platform Linter! |

| Date | Description |
|---|---|
| Mar 18, 2026 | **Added Mandatory Determinism Rule (Step 1a).** Prohibits the use of non-deterministic language such as "or," "recommended," "should," and "etc." to ensure the prompt acts as a strict state machine with a single execution path. |
| Mar 18, 2026 | **Updated Step 1b: Requirement Philosophy.** Established a clear hierarchy for "Overly Specific" (Critical Fail) vs. "Nice to Have" (Rubric Only) to prevent task overfitting and ensure implementation-agnostic verification. |
| Mar 18, 2026 | **Introduced 🧪 STEP 1c: Checking Your Re-written prompt.** Added mandatory auditing protocols for Linguistic Determinism and Structural Compliance, incorporating the promptchecker.md resource for Expected Interface validation. |
| Mar 13, 2026 | Any type of coding agent is ok to be used in this project! See Coding Agent Allowed.<br>We recommend OpenCode over the use of Cursor due to cost effectiveness (**https://opencode.ai/**). Link it to your favorite LLM account to get started! |
| Mar 11, 2026 | When building your website, ensure the UI meets professional freelance standards and all user flows are fully functional.<br>Even if UI design is not a strong requirement if the seed prompt didn't mention it, please make sure the website at least looks good & acceptable for freelance work standards! |
| Mar 10, 2026 | **Visual Asset Compliance**<br>• **Commercially Free Assets:** If your task involves using any visuals including **icons, stock images, illustrations, or fonts**, you must ensure they are **100% commercially free to use**, and **100% ok to use their content for AI and ML development.**<br>• **License Verification:** Do not include assets that require a paid license, attribution (unless specified), or have restrictive copyright tags.<br>• **Example Sources:** Use resources like **Google Fonts, Lucide/Heroicons, or Pexels** (where applicable) to ensure the "Golden Patch" is legally deployable.<br>• If you are not sure, please only use an image placeholder in your code!<br>**!!Do NOT USE any content from Unsplash! !!** |
| | **Don't be afraid to feed the QC Spec doc downloaded copy to Cursor Agent to help you check your prompt, rubrics, golden solution, and tests as well!** |

| Date | Description |
| --- | --- |
| Mar 18, 2026 | **Added Mandatory Determinism Rule (Step 1a).** Prohibits the use of non-deterministic language such as "or," "recommended," "should," and "etc." to ensure the prompt acts as a strict state machine with a single execution path. |
| Mar 18, 2026 | **Updated Step 1b: Requirement Philosophy.** Established a clear hierarchy for "Overly Specific" (Critical Fail) vs. "Nice to Have" (Rubric Only) to prevent task overfitting and ensure implementation-agnostic verification. |
| Mar 18, 2026 | **Introduced 🧪 STEP 1c: Checking Your Re-written prompt.** Added mandatory auditing protocols for Linguistic Determinism and Structural Compliance, incorporating the promptchecker.md resource for Expected Interface validation. |
| Mar 13, 2026 | Any type of coding agent is ok to be used in this project! See Coding Agent Allowed.<br>We recommend OpenCode over the use of Cursor due to cost effectiveness (**https://opencode.ai/**). Link it to your favorite LLM account to get started! |
| Mar 11, 2026 | When building your website, ensure the UI meets professional freelance standards and all user flows are fully functional.<br>Even if UI design is not a strong requirement if the seed prompt didn't mention it, please make sure the website at least looks good & acceptable for freelance work standards! |
| Mar 10, 2026 | **Visual Asset Compliance**<br>• **Commercially Free Assets:** If your task involves using any visuals including **icons, stock images, illustrations, or fonts**, you must ensure they are **100% commercially free to use**, and **100% ok to use their content for AI and ML development.**<br>• **License Verification:** Do not include assets that require a paid license, attribution (unless specified), or have restrictive copyright tags.<br>• **Example Sources:** Use resources like **Google Fonts, Lucide/Heroicons, or Pexels** (where applicable) to ensure the "Golden Patch" is legally deployable.<br>• If you are not sure, please only use an image placeholder in your code!<br>**!!Do NOT USE any content from Unsplash! !!** |
| Feb 25, 20... | Unit Tests Should NOT be Overly Specific!!<br>Test Driven Development (TDD) Approach |
| Feb 18, 2026 | Updated Docker File, Added Validation Script, how to use it, and what you can |

| Date | Description |
|---|---|
| Mar 18, 2026 | **Added Mandatory Determinism Rule (Step 1a).** Prohibits the use of non-deterministic language such as "or," "recommended," "should," and "etc." to ensure the prompt acts as a strict state machine with a single execution path. |
| Mar 18, 2026 | **Updated Step 1b: Requirement Philosophy.** Established a clear hierarchy for "Overly Specific" (Critical Fail) vs. "Nice to Have" (Rubric Only) to prevent task overfitting and ensure implementation-agnostic verification. |
| Mar 18, 2026 | **Introduced 🧪 STEP 1c: Checking Your Re-written prompt.** Added mandatory auditing protocols for Linguistic Determinism and Structural Compliance, incorporating the promptchecker.md resource for Expected Interface validation. |
| Mar 13, 2026 | Any type of coding agent is ok to be used in this project! See Coding Agent Allowed.<br>We recommend OpenCode over the use of Cursor due to cost effectiveness (**https://opencode.ai/**). Link it to your favorite LLM account to get started! |
| Mar 11, 2026 | When building your website, ensure the UI meets professional freelance standards and all user flows are fully functional.<br>Even if UI design is not a strong requirement if the seed prompt didn't mention it, please make sure the website at least looks good & acceptable for freelance work standards! |
| Mar 10, 2026 | **Visual Asset Compliance**<br>• **Commercially Free Assets:** If your task involves using any visuals including **icons, stock images, illustrations, or fonts**, you must ensure they are **100% commercially free to use**, and **100% ok to use their content for AI and ML development.**<br>• **License Verification:** Do not include assets that require a paid license, attribution (unless specified), or have restrictive copyright tags.<br>• **Example Sources:** Use resources like **Google Fonts, Lucide/Heroicons, or Pexels** (where applicable) to ensure the "Golden Patch" is legally deployable.<br>• If you are not sure, please only use an image placeholder in your code!<br>**!!Do NOT USE any content from Unsplash! !!** |
| | change with it.<br>Added what should be included in your re-written prompt. |
| Feb 2, 2026 | Adding "Expected Interface" to the rewritten prompt |

# 🎯 Project Goal

The objective is to generate **high-quality, verified software solutions** for freelance-style tasks starting from a blank slate. You will transform a raw task description into a structured agent prompt, develop a "from-scratch" functional solution (**Golden Patch**), and provide a dual-layer verification suite (automated F2P tests + expert rubric).

---

# 📖 Table of Contents

# ✅ Coding Agent Allowed

To assist with your development workflow in this project, you are **eligible and encouraged to create an account** with **Cursor, OpenCode, or your preferred coding agent** if you do not already have one.

**OpenCode**

- Website: https://opencode.ai/
- If this is your **first time working on a task**, we recommend starting with **OpenCode**.
- You can **link it to your preferred LLM Pro account** to get started.

- **OpenCode is free to use.**

**Cursor (Reimbursement Policy)**

- Subscription costs for **Cursor** will be **reimbursed only after the successful completion of your first task**, provided the task is **not considered spam or low quality** (QC score of **1/5 or 2/5**).
- Please start with the **$20 subscription plan** initially.
- **Higher-tier subscription reimbursement requirements:**
  - **$200+ membership**
    - Must complete **≥ 30 tasks**
    - Maintain an **average QC score ≥ 3/5**
    - Must be achieved **within 30 days of purchase**
  - **$60–$70 membership**
    - Must complete **≥ 15 tasks**
    - Maintain an **average QC score ≥ 3/5**
    - Must be achieved **within 30 days of purchase**
- **Important Note:**
  - Ensure that the **viewer permission for your receipt screenshot is set to public**, otherwise the screenshot cannot be accessed for verification.
  - A **reimbursement form will be provided later** if you:
    - Pass all screening requirements
    - Successfully complete your **first high-quality task**
    - Meet the **membership tier requirements listed above**

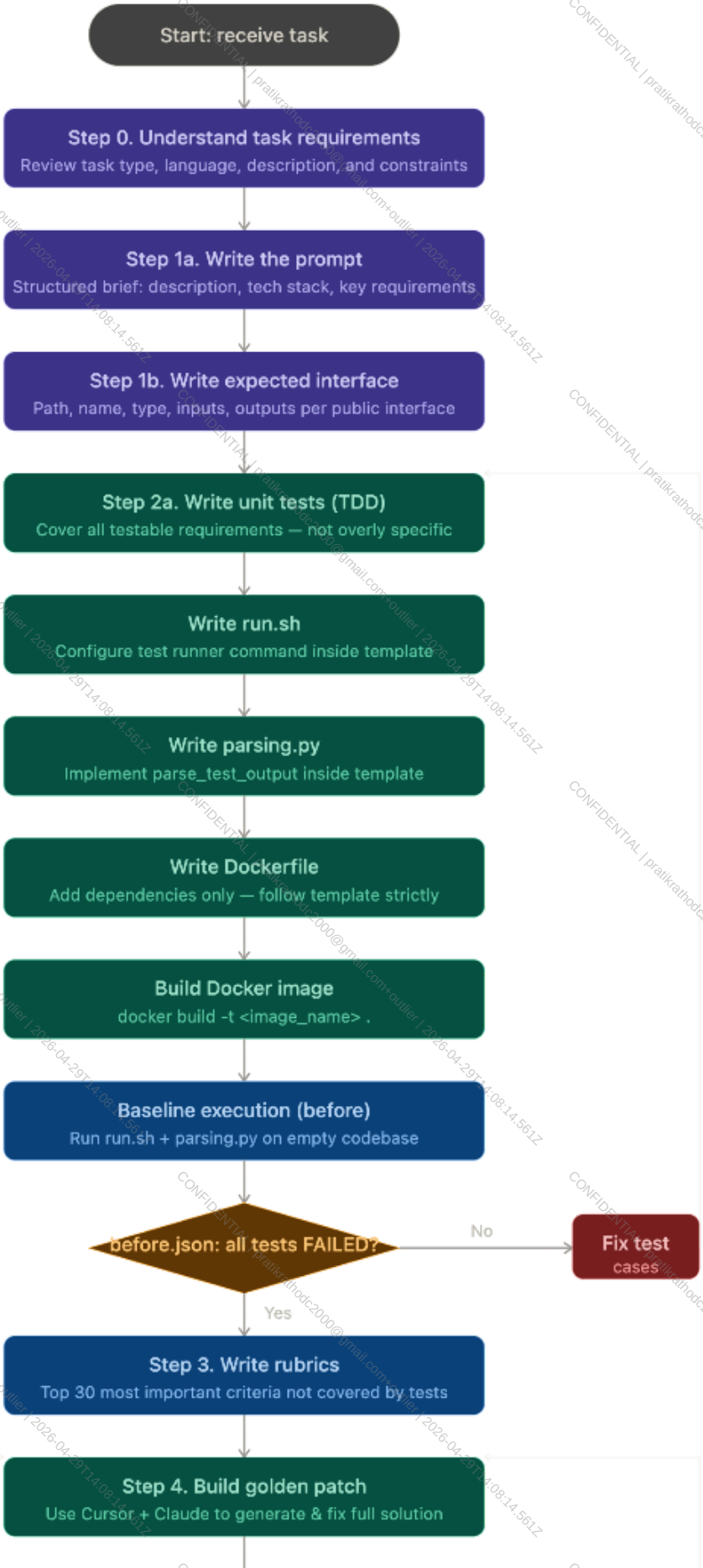
---

# 🔄 Task Workflow

🐸**Follow these steps in order to ensure a successful submission:**🐸

1. Review the **task description** (usually a job description from a freelance website) and note down the requirements, constraints, language, and expected output.
2. **Build a prompt** that consists of all the necessary end-to-end functionality and overall software system design in order to achieve all the requirements in the task description.
3. Make sure your prompt has the **Expected Interface** section included, and that it covers every newly introduced **file**, **function**, or **class** that an external application or test suite will interact with, focusing only on public interfaces and not helper functions.

4. Use **Test Driven Development (TDD)** by leveraging **Cursor agent + Claude** to build the set of **unit tests** that cover all the requirements testable by unit tests **without** being **overly specific**.
    a. Check the set of unit tests and make sure they **cover all the requirements** in your **prompt** that are testable by unit tests **without** being **overly specific**, using the system prompt if needed to validate coverage.
5. Write **run.sh** to configure the test runner command inside the template.
6. Write **parsing.py** to implement parse_test_output inside the template.
7. Add required dependencies in the **Dockerfile** by following the template strictly **without** using the **COPY command**.
8. **Baseline Execution (Before):** Build the **Docker image** and run baseline execution using **run.sh** and **parsing.py** on an **empty codebase**.
    a. **Goal:** Ensure the resulting **JSON** shows every unit test as **FAILED** (not **ERRORED**), and fix test cases if this condition is not met.
9. Write the **rubrics** covering the top most important requirements that are not evaluated through unit tests.
    a. **Goal:** Ensure each criterion is **atomic**, **verifiable**, and **positively phrased**, uses only weights **1**, **3**, or **5**, and collectively evaluates **instruction following, correctness, quality, clarity, and efficiency.**
10. Ask the agent to **build the complete solution** of your prompt and fix the agent's solution to create your **Golden Patch**.
11. **Verification Execution (After):** Apply the **Golden Patch** and re-run both **run.sh** and **parsing.py** scripts for verification execution.
    a. **Goal:** Ensure the resulting **JSON** now shows the same set of tests as all **PASSED**, and fix the Golden Patch if needed.
12. Rate the **rubrics** against the **Golden Patch** by evaluating each criterion using the golden response, and fix the Golden Patch or rubrics if required.
13. Run the **validation script** to confirm **before.json** shows all **FAIL** and **after.json** shows all **PASS**.
14. **Check** the **folder structure** and **output**, and fix any issues if necessary.
15. Upload the finalized files including **Golden Patch (codebase.zip)**, Dockerfile, **run.sh**, **parsing.py**, and **tests.zip**, and submit.
16. SUBMIT! 🎉🐸


Start: receive task
Step 0. Understand task requirements
Review task type, language, description, and constraints
Step 1a. Write the prompt
Structured brief: description, tech stack, key requirements
Step 1b. Write expected interface
Path, name, type, inputs, outputs per public interface
Step 2a. Write unit tests (TDD)
Cover all testable requirements — not overly specific
Write run.sh
Configure test runner command inside template
Write parsing.py
Implement parse_test_output inside template
Write Dockerfile
Add dependencies only — follow template strictly
Build Docker image
docker build -t <image_name> .
Baseline execution (before)
Run run.sh + parsing.py on empty codebase
before.json: all tests FAILED?
No
Fix test cases
Yes
Step 3. Write rubrics
Top 30 most important criteria not covered by tests
Step 4. Build golden patch
Use Cursor + Claude to generate & fix full solution

---

## 📚 Resources

- Onboarding Video
- General Onboarding
- What to modify in Dockerfile

**Off-Platform Linter/System Prompt to check Golden Patch, Unit Tests, Rubrics: (02/27)**

- Tutorial on How to Use it
- System Prompt Folder
- Another System Prompt Folder

> **Important Tip**
> Don't be afraid to feed the QC Spec doc downloaded copy to the Cursor Agent to help you check your prompt, rubrics, golden solution, and tests as well!

> **Warning**
> Please note that these system prompts (off-platform linters) should be only used as a guide and should not be treated as a source of truth. You are responsible for the correctness of your solution.

## ✏️ STEP 0: Understanding Task Requirements

Before rewriting the prompt, carefully review the **Task Type**, **Task Coding Language**, **Short Description**, and **Task Description** provided in the seed task. This step ensures you clearly understand the **scope, constraints, and goals** before transforming the task into a structured prompt.

### Key Sections to Review

**Task Type**
Defines the general **category of the application** (e.g., operations tool, web platform, data pipeline, mobile app). This helps you understand the expected **problem domain and functionality**.

**Task Coding Language**
Specifies whether the implementation must use a **specific programming language** or if **any language is allowed**. If unrestricted, you may define an appropriate **tech stack** in your rewritten prompt.

**Short Description**
This section defines global constraints that apply to all tasks and should be preserved when rewriting the prompt.

**Key rules:**

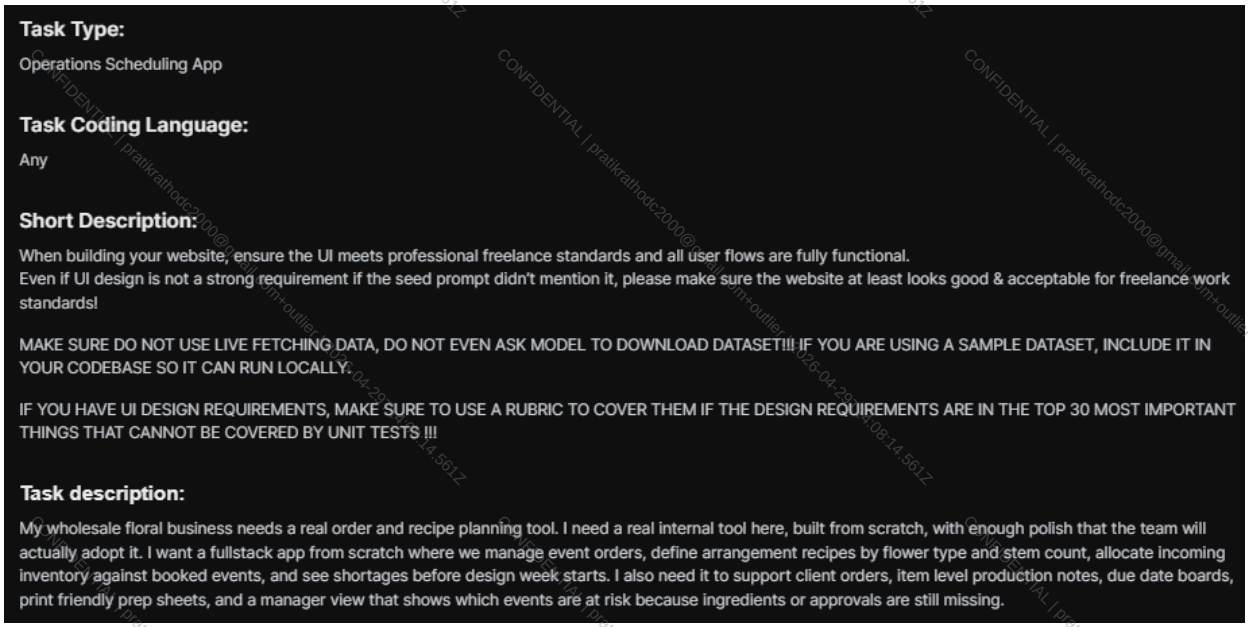
- Maintain professional freelance development standards.
- If the task includes a UI, it must be clean, usable, and functional.
- All core user flows must work correctly.
- No live data fetching or external dataset downloads are allowed.
- Any required dataset must be included locally in the repository.
- Unit tests should cover functional requirements, while rubrics should cover UI/design aspects that cannot be tested automatically (limit to the top 30 most important criteria).

**Task Description**
This is the **most important input**. It is typically written as a **high-level client-style brief** describing what the system should do.

Your responsibility is to transform this into a **clear, structured, and implementation-ready prompt** by:

- Making requirements **specific and unambiguous**
- Expanding vague descriptions into **concrete functionality**
- Ensuring **logical consistency** across all requirements
- Removing or resolving **contradictions**
- Filling in **reasonable technical details** when necessary

The final prompt should give the AI agent **enough clarity to implement the system and you can generate correct tests without guessing missing requirements.**.




## Visual Asset Compliance

- **Commercially Free Assets:** If your task involves using any visuals including **icons, stock images, illustrations, or fonts**, you must ensure they are **100% commercially free to use**, and **100% ok to use their content for AI and ML development.**
- **License Verification:** Do not include assets that require a paid license, attribution (unless specified), or have restrictive copyright tags.
- **Example Sources:** Use resources like **Google Fonts, Lucide/Heroicons, or Pexels** (where applicable) to ensure the "Golden Patch" is legally deployable.
- If you are not sure, please only use an image placeholder in your code!

**!!Do NOT USE any content from Unsplash! !!**

# 📝 STEP 1a: Prompt Generation

> 💡 **Important Tip**
> **You HAVE TO include the complete Expected Interface section in your re-written prompt!** Failure in doing so will directly result in a bad quality task.

The first step is to transform a raw task description into a structured prompt. This prompt functions as a **Job Application/Freelance Brief** that an AI agent will use to come up with a solution.

- **Clarity:** Ensure the goal and tech stack are unambiguous.
- **Context:** Include all necessary information for the agent to create the correct tests and solution (Golden Patch).
- **Expected interface:** Define every newly introduced file, function, or class that an external application (i.e. not the same codebase) or test suite will interact with. Do not need to flag for helper functions or fields from 3rd party libraries **(02/24)**. It is only about end-to-end functionality, and the overall architectural design of the software.
- **NOTE:** There should be an expected interface for every test case
    - **Path:** `[Exact file path as it appears in the intended structure]`
    - **Name:** `[Class.method or function name]`
    - **Type:** `[e.g., class, method, function, or interface]`
    - **Input:** `[Parameters and types, e.g., chunk: GlibcChunk]`
    - **Output:** `[Return type, e.g., None or Promise<void>]`
    - **Description:** `[Briefly describe observable side effects or behavior asserted by tests]`

    **Language-Specific Fields (As applicable):**
    - **Inheritance:** `extends <Base>; implements <IfaceA, IfaceB>` (TypeScript/Java)
    - **Embedding / Implements:** `embeds <TypeA>; implements <IfaceA, IfaceB>` (Go)
    - **Bases / Overrides:** `bases: <BaseA, BaseB>; overrides: <Base.method>` (Python)
    - **Annotations / Decorators:** `@Override, @Inject, @dataclass, @cached_property, @sealed`

## 🧩 Common Prompt Structure Pattern

The problem statements follow a consistent markdown-based template with these recurring sections:

### Core Sections (most frequent)

| Section | Frequency | Description |
|---|---|---|

| | | |
|---|---|---|
| Title (# Title) | 14/20 | Short project name (e.g., "Build Complete Thai Restaurant Website") |
| Description / Context | 18/20 | The original freelancer-style brief explaining what the client needs |
| Tech Stack | 12/20 | Specified languages, frameworks, databases, testing tools |
| **Expected Interface** | **20/20** | The most critical section — defines every file, function, class, and API endpoint that tests will interact with |
| Current State | 14/20 | Almost always "Empty repository with test files only" |
| Required Implementation | 10/20 | Bullet list of what to build (features, components, behaviors) |
| Key Requirements | 8/20 | Numbered functional requirements, sometimes with subsections |

## Typical Ordering (two main patterns)

- Pattern A (most common — 12/20):

```python
# Title
## Description / Context
## Tech Stack
## Key Requirements (with ### subsections)
## Expected Interface (with ### per function/class/endpoint)
## Current State
```

- Pattern B (for larger/more detailed tasks — 8/20):

```python
# Title
## Description
## Current State
## Required Implementation
## Expected Interface (with ## and ### groupings)
## Deliverables / Acceptance Criteria
```

## Expected Interface Format (universal across all)

The interface should represent the natural entry point a user or dependent system would interact with. It must never dictate internal architecture, modularization, or helper logic. Learn more about Expected Interface here.

Every entry in the Expected Interface section follows this per-item schema:

```python
- Path: <exact file path>
- Name: <Class.method or function or endpoint>
- Type: <class | function | method | API Endpoint | React Component | interface | Prisma Model | ...>
- Input: <parameters and types>
- Output: <return type or HTTP response>
- Description: <what it does and what tests verify>
```

Some tasks add language-specific fields like `Annotations: @override` (Flutter/Dart) or include `testID` mappings (React Native), TypeScript type definitions inline, or Prisma schema field requirements.

## Key Observations

- Expected Interface is always embedded inside the problem_statement — it's not a separate column. It makes up 30-70% of the total content.
- Granularity varies by complexity — simple tasks have 4-6 interface entries; complex ones (Tasks 12, 19, 20) have 20-40+.

- The intro paragraph is the "task_description" — the casual, 1-3 sentence client brief that appears before the structured sections.
- "Current State" is almost always "Empty repository with test files only" — establishing that this is a greenfield build.

## The Determinism Rule (Mandatory)

To ensure the Golden Patch is reproducible and verifiable, your rewritten prompt **MUST** act as a strict state machine. It must enforce a single, predictable execution path. You must avoid all forms of linguistic non-determinism that allow the AI agent to "choose" its own implementation details.

**Danger Zone**

**Any prompt containing optionality, soft constraints, or open-ended scopes will be immediately disabled.**

- **Mandatory Phrasing:** Use direct, imperative commands such as "You **MUST**," "Implement," or "Create".
- **Forbidden Language:** Do not use "or," "alternatively," "either X or Y," "recommended," "should," "etc.," or "you can" until and **unless the Task Description specifies it.**
- **Strict Boundaries:** Do not use "something like" or "relevant technologies." Define an exhaustive, finite list of every tool and technology required.

## Rewritten Example:

***Task description:***

**Task Type:**

CLI Tool

**Task Coding Language:**

Any

- if your Coding Language is 'Any', do NOT write 'ANY' in your prompt please!!! You have to choose a specific tech stack in your prompt that is not contradicting with requirements in Task Description below.

  **Also, PLEASE DO NOT INCLUDE UNIT TEST & RUBRIC INSTRUCTIONS IN THE PROMPT!**

**Short Description:**

MAKE SURE DO NOT USE LIVE FETCHING DATA, DO NOT EVEN ASK MODEL TO DOWNLOAD DATASET!!! IF YOU ARE USING A SAMPLE DATASET, INCLUDE IT IN YOUR CODEBASE SO IT CAN RUN LOCALLY.

**Task description:**

Med student here, I make hundreds of flashcards for exam prep. I want a terminal-based flashcard tool. Load a deck from a simple text file (question on one line, answer on the next, blank line between cards). Show me the question, I press enter to reveal the answer, then I mark it as correct or wrong. At the end show my score. Shuffle the deck each time. Simple spaced repetition where missed cards come back more often would be amazing but not required.

***Re-written prompt:***

```python
# Title
Terminal-Based Flashcard Application

## Description / Context
Build a terminal-based flashcard tool for medical exam
preparation. The tool loads a deck from a simple text file
(question on one line, answer on the next, blank line between
cards). It displays the question, waits for Enter to reveal the
answer, then lets the user mark each card as correct or wrong. At
the end, it shows the session score. The deck is shuffled each
time. All data must be bundled in the codebase—no live fetching,
no external dataset downloads. The application must run fully
offline and locally.

## Tech Stack
- Language: Python
- Data: Local text file included in repository
- No external APIs or dataset downloads

## Key Requirements

### Data Requirements
```

- Deck must be loaded from a local text file included in the repository
- Do not fetch from external URLs or download any dataset
- Application must run completely locally without network access for data

### Deck File Format
- One question per line
- Answer on the immediately following line
- Blank line between cards (no blank line between a question and its answer)

- Example:

```
What is the capital of France?
Paris

What is 2 + 2?
4
```

### Core Behavior
1. Load deck from file path (e.g., data/sample_deck.txt)
2. Shuffle the deck at the start of each session
3. For each card: show question → wait for Enter → reveal answer → user marks the card as correct or wrong (the exact key choice, e.g., c/w or y/n, is left to the implementer)
4. At session end: display score (e.g., "5/10 correct" or "Score: 50%")

## Expected Interface
### Deck Loader
- Path: flashcard.py
- Name: load_deck
- Type: Function
- Input: filepath: str
- Output: list[tuple[str, str]]

- Description: Parses the deck file and returns a list of (question, answer) pairs. The file format is: question on one line, answer on the immediately following line, one or more blank lines separating each card pair. Trailing blank lines at EOF must be tolerated. Lines must be stripped of leading/trailing whitespace. If the file contains an odd number of non-blank lines (i.e., a question with no corresponding answer), the function should raise a ValueError or equivalent error to signal malformed input.

### Entry Point
- Path: flashcard.py
- Name: main
- Type: Function
- Input: filepath: str
- Output: None
- Description: Runs the full flashcard session for the given deck file. Shuffles the deck, iterates through cards showing the question, waits for Enter to reveal the answer, collects the user's correct/wrong marking for each card, and prints the final score at the end.

### CLI Execution
- Path: flashcard.py
- Name: flashcard.py (module-level execution)
- Type: Script / CLI entry point
- Input: One positional command-line argument: the path to the deck file (e.g., `python flashcard.py data/sample_deck.txt`)
- Output: None (interactive terminal session with final score printed to stdout)
- Description: When executed as a script, flashcard.py must read the deck file path from the command-line arguments and invoke `main(filepath)`. The script must exit cleanly after the session ends.

### Bundled Deck File

- Path: data/sample_deck.txt
- Name: sample_deck.txt
- Type: File
- Input: N/A (static file read by load_deck)
- Output: N/A (static file read by load_deck)
- Description: A deck file bundled with the repository containing at least 5 flashcard entries (medical or general knowledge). Must conform to the deck file format: question line, answer line on the next line, blank line between card pairs. This file must exist at the specified path so that tests can invoke load_deck("data/sample_deck.txt") and receive a non-empty list.

### Dependency Manifest
- Path: requirements.txt
- Name: requirements.txt
- Type: File
- Input: N/A (static file)
- Output: N/A (static file)
- Description: Must exist at the repository root. Lists any third-party Python dependencies required to run the application. May be empty if the implementation uses only the Python standard library.

## Current State
Empty repository with test files only.

## Required Implementation
- Implement `load_deck(filepath)` to parse the deck file format and return a list of (question, answer) pairs; tolerate multiple consecutive blank lines and trailing blank lines at EOF; strip whitespace from each line; raise an error on malformed input (odd number of non-blank lines)
- Implement `main(filepath)` that runs the full session; the script must be executable from the command line as: `python flashcard.py data/sample_deck.txt`
- Shuffle the deck at the start of each session

```
- Interactive loop: display question → wait for Enter → display
answer → accept user input marking the card correct or wrong
(exact prompt/key choice is flexible)
- Display the final score at the end of the session (e.g., "5/10
correct" or "Score: 50%" — exact format is flexible as long as
both the count of correct answers and total cards are
communicated)
- Include `data/sample_deck.txt` with at least 5 sample
flashcards (medical or general knowledge) in the correct format
- Include `requirements.txt` (can be empty if only stdlib is
used)

## Optional Enhancements
- Simple spaced repetition within a session: missed cards
reappear more often during the same session. This is not required
and must not be tested for.

## Deliverables
- flashcard.py — Contains load_deck and main entry point
- data/sample_deck.txt — Local deck file with sample cards
(included in codebase)
- requirements.txt — Required (can be empty if stdlib only)
- Working CLI that runs locally; all data bundled; no external
fetch or download
```

## 🧪 Step 1b: Prompt Requirements

Your prompt Requirements should be separated into 2 main categories:

1. **Requirements from your prompt that can be tested with a unit test suites that are NOT overly-specific**
    a. Overly specific here means that your tests should not penalize another possible golden solution that satisfies the prompt you wrote! Check Section 2b.

2. **Requirements from your prompt that can only be tested with rubrics that are either:**

a. Too overly specific to be covered by unit tests
b. Can only be covered by rubrics
c. Check Section 3a for more clarity on how to write rubrics.

For any requirements in number 2, you only need to cover up to **TOP 30 MOST IMPORTANT** explicitly mentioned criteria in the rubrics part that are not tested by the unit tests.

## Requirement Philosophy: Specificity vs. Nice-to-Have

To avoid "**Overfitting**" (where a test or rubric is too rigid and incorrectly rejects a valid solution), you must categorize your requirements according to the following logic:

| Category | Definition | Allowed In |
|---|---|---|
| **Overly Specific** | Verifying a detail that is **never** explicitly or implicitly mentioned in the prompt (e.g., enforcing a specific internal folder name or private method name you used in your Golden Patch). | **NONE** (This is a Critical Failure). |
| **Nice to Have (Implicit)** | Verifying a standard expectation that is **implicitly** mentioned (e.g., a "professional UI" implies consistent button padding even if not explicitly defined). | **Rubrics Only** (Weight 1). |
| **Nice to Have (Explicit)** | Verifying a feature that the prompt explicitly labeled as "Optional," "Recommended," or "Nice to Have". | **Rubrics Only** (Weight 1). |

**IMPORTANT INSTRUCTION**

🛑If you have a **frontend development / Fullstack development** task, make sure to use Instruction Following (IF) rubrics to cover any UI Design requirements in your re-written prompt!

🛑If you do NOT want to cover so many UI Design requirements, DO NOT include design asks in your re-written prompt!

🛑When building your website, ensure the UI meets professional freelance standards and all user flows are fully functional.

🛑Even if UI design is not a strong requirement if the seed prompt didn't mention

> it, please make sure the website at least looks good & acceptable for freelance work standards!

## 🧪Step 1c: Checking Your Re-written prompt

Before proceeding to Unit Test creation or building the Golden Patch, you must validate that your re-written prompt acts as a **strict state machine**. A poorly structured or ambiguous prompt leads to non-deterministic solutions, inconsistent test passes, and will ultimately result in task rejection.
Follow these two mandatory checks to ensure your prompt is implementation-ready:

### Check 1: Linguistic Non-Determinism (Linter)

Feed the system prompt below to Cursor to audit your prompt for "Logic Traps" like optionality, soft constraints, or vague boundaries.

**Goal:** Your prompt must achieve a **PASS** status. If any "Optionality" or "Soft Constraint" traps are found, you must replace permissive language (e.g., "you can") with direct imperative commands (e.g., "You MUST").

Shell

```
You are a Principal Prompt Architect. Your sole objective is to
audit Contributor (CB) prompts for LINGUISTIC NON-DETERMINISM.

A high-quality prompt must act as a strict state machine. It must
enforce a single, predictable execution path. You will fail any
prompt that contains optionality, soft constraints,
permission-based phrasing, or open-ended scopes. If a requirement
is not an absolute mandate (MUST), it is a Logic Trap and must be
flagged for removal.

1. AUDIT DIMENSION: THE OPTIONALITY TRAP (Branching Paths)
[Fail - Multiple Choice]: The prompt gives the AI a choice
without a strict programmatic condition for how to choose.
* Triggers: "or", "alternatively", "either X or Y", "you can
choose".
* Example Trap: "You can use React or Vue." (This guarantees
varying outputs across iterations).
```

* Rule: The prompt must dictate ONE exact path, or provide a strict IF/THEN condition based on input variables.

2. AUDIT DIMENSION: THE SOFT CONSTRAINT TRAP (Weak Modifiers)
[Fail - Suggestion over Command]: The prompt uses language that makes a requirement optional, leading the AI to randomly ignore or apply it.
* Triggers: "should", "preferably", "ideally", "if possible", "recommended", "nice to have", "try to".
* Example Trap: "It is recommended to use TypeScript, but not required."
* Rule: A requirement is either a "MUST" / "SHALL" or it must be deleted entirely.

3. AUDIT DIMENSION: THE OPEN-ENDED SCOPE TRAP (Vague Boundaries)
[Fail - Unbounded Lists]: The prompt asks the AI to extrapolate beyond a defined set, causing unpredictable hallucination of scope.
* Triggers: "etc.", "similar tools", "something like", "relevant technologies", "and others".
* Example Trap: "Write the backend using Express, Fastify, etc."
* Rule: The prompt must define an exhaustive, finite list or name a singular exact technology.

4. AUDIT DIMENSION: THE PERMISSION TRAP (Passive Voice)
[Fail - Capability vs. Instruction]: Stating what the AI *is allowed* to do rather than what it *must* do.
* Triggers: "you can", "you may", "feel free to", "you have the option to".
* Example Trap: "You can use this coding language..."
* Rule: Replace permissive language with direct imperative commands (e.g., "Use this coding language...").

5. MANDATORY AUDIT PROTOCOLS

```
* The Keyword Sweep: You must explicitly scan the text for the
forbidden words listed above. If found, highlight the exact
sentence.
* The "What If" Test: For every sentence, ask: "Could an LLM read
this and legally decide NOT to do it?" If yes, the prompt is
non-deterministic and fails.

6. INPUT
Contributor Prompt to Audit: {{promptText}}

7. OUTPUT SPECIFICATION (JSON ONLY)
{
  "status": "PASS" | "FAIL",
  "determinism_score": "0-100",
  "violations": [
    {
      "dimension": "Optionality Trap" | "Soft Constraint Trap" |
"Open-Ended Scope Trap" | "Permission Trap",
      "severity": "Critical",
      "offending_text": "The exact sentence containing the
non-deterministic language.",
      "forbidden_trigger": "The specific word(s) flagged (e.g.,
'recommended but not required', 'or', 'you can').",
      "feedback": "Explain why this creates a non-deterministic
execution path for the model.",
      "remediation": "Provide the exact rewritten sentence
enforcing strict determinism (e.g., 'Change to: You MUST use
[Specific Tech].')."
    }
  ]
}
```

## Check 2: Structural & Guideline Compliance

Use the linter below to verify that your prompt follows the required **Pattern A or B** structure and contains a complete **Expected Interface** for every test case.

To perform this check, refer to the **System Prompt Folder** located in the **Resources** section of this document. Specifically, utilize the content within the **promptchecker.md** file.

**Goal:** Ensure 100% alignment with the **Expected Interface** schema (Path, Name, Type, Input, Output, Description) and verify that the **Tech Stack** is unambiguous.

```shell
---
alwaysApply: true
---

Prompt Engineering: Re-write the task description into a
structured entry point
that guides an AI agent to build the solution from scratch.

---

## GOAL

The first step is to transform a raw task description into a
structured prompt. This prompt
functions as a Job Application/Freelance Brief that an AI agent
will use to come up with a
solution.

---

## RULES TO FOLLOW TO MAKE THE PROMPT

**Clarity:** Ensure the goal and tech stack are unambiguous.

**Context:** Include all necessary information for the agent to
create the correct tests
and solution (Golden Patch).
```

**Expected interface:** Include a section within the rewritten prompt to define every
newly introduced file, function, or class that an external module or test suite will
interact with. You should include:

- **NOTE:** There should be an expected interface for every test case

  - Path: [Exact file path as it appears in the intended structure]
  - Name: [Class.method or function name]
  - Type: [e.g., class, method, function, or interface]
  - Input: [Parameters and types, e.g., chunk: GlibcChunk]
  - Output: [Return type, e.g., None or Promise<void>]
  - Description: [Briefly describe observable side effects or behavior asserted by tests]

**Language-Specific Fields (as applicable):**

  - **TypeScript/Java:** Inheritance: extends <Base>; implements <IfaceA, IfaceB>
  - **Go:** Embedding / Implements: embeds <TypeA>; implements <IfaceA, IfaceB>
  - **Python:** Bases / Overrides: bases: <BaseA, BaseB>; overrides: <Base.method>
  - **Annotations / Decorators:** @Override, @Inject, @dataclass, @cached_property, @sealed

---

## Common Prompt Structure Pattern

### Typical Ordering (two main patterns)

**Pattern A** (most common – 12/20):

1. # Title
2. ## Description / Context
3. ## Tech Stack
4. ## Key Requirements (with ### subsections)
5. ## Expected Interface (with ### per function/class/endpoint)
6. ## Current State

**Pattern B** (for larger/more detailed tasks – 8/20):

1. # Title
2. ## Description
3. ## Current State
4. ## Required Implementation
5. ## Expected Interface (with ## and ### groupings)
6. ## Deliverables / Acceptance Criteria

### Expected Interface Format (universal across all)

Every entry in the Expected Interface section follows this per-item schema:

- **Path:** <exact file path>
- **Name:** <Class.method or function or endpoint>
- **Type:** <class | function | method | API Endpoint | React Component | interface | Prisma Model | ...>
- **Input:** <parameters and types>
- **Output:** <return type or HTTP response>
- **Description:** <what it does and what tests verify>

Some tasks add language-specific fields like Annotations: @override (Flutter/Dart) or include testID mappings (React Native), TypeScript type definitions inline, or Prisma schema field requirements.

---

### EXAMPLE

**Task description:**

Need experienced designer/developer for website to show menu and hours for our
Aroy Dee Thai Restaurant customer so the customer can easily see all info.
Please create the website and logo. Link to menu. Hours: 9am-10pm Tue-Sun,
closed Mon.

**Re-written prompt:**

Need experienced designer/developer for website to show menu and hours for our
Aroy Dee Thai Restaurant customer so the customer can easily see all info.
Please create the website and logo. Link to menu. Hours: 9am-10pm Tue-Sun,
closed Mon.

# Title
Build Complete Thai Restaurant Website

## Description
Need a complete React-based restaurant website for Aroy Dee Thai built from
scratch. The site currently has no implementation and needs full development of
all components, styling, and functionality.

## Expected Interface

- **Path:** src/components/MenuSection.js
- **Name:** MenuSection

- **Type:** React Component
- **Input:** props: { categoryName: string, items: Array }
- **Description:** A public component that renders a collapsible list of menu items. Bases / Overrides: overrides: React.Component

## Current State
Empty repository with test files only.

## Required Implementation
Build a complete mobile-first Thai restaurant website with React that passes
all 72 comprehensive tests covering rendering, content accuracy, interactions,
menu structure, visual design, and component functionality.

- Create complete React application structure with App component, MenuSection
  component, and MenuItem component
- Implement collapsible menu sections with toggle functionality
- Create 13 menu categories with full menu items, prices, and descriptions
- Design Thai-inspired aesthetic with custom SVG decorations (logo, dividers,
  ornaments)
- Implement responsive CSS that works on mobile (≤480px) and desktop (≥768px)
  viewports
- Display restaurant information (address, hours, spice warning)
- Create elegant menu header section
- Ensure all 72 tests pass covering rendering, content, interaction, menu
  accuracy, and visual design

> **Warning**
> Please note that these system prompts (off-platform linters) should be only used as a guide and should not be treated as a source of truth. You are responsible for the correctness of your solution.

> **STOP!**
> This is the end of **STEP 1a: Prompt Re-writing, STEP 1b: Prompt Requirement and Step 1c: Checking Your Re-written prompt** section! If you have any questions, please go to the war room in your dashboard and we will have QMs to help you there!

---

# STEP 2: Crafting Test Cases

The goal of this project is to create verified, functional software. To achieve this, your test suite must prove that the code **works**, not just that the code **exists**.

## 1. Static vs. Functional Testing

We have a zero-tolerance policy for "Static String Matching" (also known as keyword testing).

- **Static Testing (FAIL):** Using scripts that "grep" or search through source files for specific words (e.g., searching `index.ts` for the word "express").
- **Functional Testing (PASS):** Using a test runner (like `pytest`) to actually execute the code, send real inputs, and assert that the outputs are correct.

## 2. The "Comment Trap"

A major weakness of static testing is that it cannot distinguish between functional logic and dead text.

> **Example:** If a test only checks for the word `"month"` to verify a date-filtering feature, a developer could simply write a comment like `// TODO: implement month filtering`. The static test would erroneously **PASS**, while the software remains broken. A functional test would catch this immediately by attempting to filter data by month and receiving an incorrect result.

## 3. The Black-Box Principle

Tests must treat your **Golden Patch** as a "Black-Box".

- **Interaction:** Tests should only interact with the solution through the **Expected Interface** (API endpoints, public class methods, or CLI commands).
- **Isolation:** Do not mock or patch internal helper functions. If the backend is built correctly, the test should be able to trigger a behavior (like a file upload) and verify the side effect (like a new record in the database) without knowing how the code achieved it.

## 4. Deterministic Outcomes

Your tests must be the ultimate "truth" for the task.

- **Fail-to-Pass Integrity:** The exact same test suite **MUST** fail on an empty codebase and **MUST** pass once the Golden Patch is applied.
- **Agnostic Verification:** A test should pass for *any* valid solution to the prompt, regardless of whether the developer used different variable names or internal file structures not specified in the prompt.

## 🎯 The "Golden Zone" of Testing

To pass QC, your tests must move beyond "looking" at code and start **executing** it.

| Feature | Overly Broad (FAIL) | Overly Specific (FAIL) | The Golden Zone (PASS) |
|---|---|---|---|
| **Verification Method** | String matching / "Grepping" for keywords like `Date` or `API`. | Mandating private method names or internal import paths. | **Functional Testing:** Making real HTTP requests or method calls to the public interface. |
| **The "Comment Trap"** | A test that passes because the word `"month"` exists in a comment, even if the code is broken. | Tests that fail if you change a variable name from `user_id` to `uid` (if the prompt didn't specify). | **Result-Oriented:** Sending input `X` and asserting that the system returns output `Y`. |

| Feature | Overly Broad (FAIL) | Overly Specific (FAIL) | The Golden Zone (PASS) |
|---|---|---|---|
| **Coverage Scope** | Only verifying that files exist or dependencies are listed. | Testing "Best Practices" or security headers not mentioned in the prompt. | **Requirement Mapping:** Every major backend requirement in your prompt has a functional test. |

---

## 💎STEP 2a: F2P Test Case Verification (Fail-to-Pass)

### Important Tip

The unit tests should NOT be overly specific to your golden solution. The reason behind it is that if we feed the same prompt you wrote to another agent, we do not want to penalize their solution if they used some other methods to also correctly solve all requirements from your prompt!

If there is something that is impossible to be covered by the prompt, use a rubric instead in the rubric section.

F2P tests are mandatory. Only skip when there would be suboptimal tests that are **too prescriptive (overly specific)** in correct implementation. If you do have test cases, there should be a "Before vs. After" comparison after the solution is implemented.

This project encourages a **Test-driven development (TDD) approach**, which means that you should pre-build the unit tests that cover all the coverable requirements in your prompt before you go on and create the golden patch. This is the best way to avoid overly specific unit tests.

Once you consolidate your prompt, you can:

1. Use cursor agent + Claude 4.6 (or any other model of your choice) to help you build the set of unit tests that covers all the backend requirements from your prompt without being overly specific first.
2. Check the set of unit tests, make sure they are of good coverage
3. **Baseline Execution (Before):** Run run.sh and parsing.py on an empty codebase.

    a. **Goal:** The resulting JSON must show specific tests as FAILED.
4. Ask the agent to build the complete solution of your prompt
5. Fix the agent's solution to create your **Golden Patch**
6. **Verification Execution (After):** Apply the **Gold Patch** and re-run both scripts.
    a. **Goal:** The resulting JSON must now show those same tests as PASSED.
7. Write the rubrics to cover the top 30 most important requirements that your unit tests cannot cover!

---

## 🛠️ Golden Patch (Dockerfile)

Ensure your environment is consistent. Copy and modify the **System Dependencies** as needed for your specific language/framework.

**Note:** You can add specific dependencies if needed

### 🐳 Docker Environment Setup & Execution Guide

This document explains how to correctly configure and run the Docker environment for your tasks in the Real Coder project.

**IMPORTANT:** It is extremely important that all instructions in this document are followed precisely. The validation script depends on a strict structure and any unintended changes may break the evaluation process.

> **SETUP DOCKER IN YOUR DEVICE**
>
> - **FOR WINDOWS**
> - **FOR MAC**
>
> **CHECK THE INSTRUCTIONS HERE**

---

### Requirements

1. **Use the provided Dockerfile template:** The provided Dockerfile template MUST be used, and can ONLY be modified for the installation of required dependencies for your solution and your tests.

   Download the Dockerfile Template here.

   **Make sure you don't:**

- Modify the structure of the file
- Remove any existing instructions
- Add unrelated layers of steps
- Change entry points or default commands

```Shell
################################################
# BASE IMAGE
################################################
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

################################################
# SYSTEM DEPENDENCIES
################################################
RUN apt-get update && apt-get install -y \
    git \
    python3 \
    python3-pip \
    python3-setuptools \
    python-is-python3 \
    unzip \
    && rm -rf /var/lib/apt/lists/*

################################################
# WORKING DIRECTORY + GIT SETUP
################################################
WORKDIR /app

RUN git init \
    && git config --global user.email "agent@example.com" \
    && git config --global user.name "Agent" \
    && echo "# Workspace" > README.md \
    && git add README.md \
    && git commit -m "Initial commit"
```

```
################################################
# EVALUATION ASSETS DIRECTORY
################################################
# Populated at RUNTIME by the evaluation script.
RUN mkdir -p /eval_assets

CMD ["/bin/bash"]
```

2. ***run.sh*** **file based on the template:** The **run.sh** file MUST be created based on the following template, so the validation can work as expected. Keep in mind that the file can only be changed within the delimited comments.

```Shell
#!/bin/bash
### COMMON SETUP; DO NOT MODIFY ###
set -e

# --- CONFIGURE THIS SECTION ---
# Replace this with your command to run all tests
run_all_tests() {
  echo "Running all tests..."
  # TODO: Run the full test suite
  # Example: cargo test --workspace --lib --no-fail-fast
}
# --- END CONFIGURATION SECTION ---

### COMMON EXECUTION; DO NOT MODIFY ###
run_all_tests
```

3. ***parsing.py*** **file based on the template:** The **parsing.py** file MUST also be based on the following template. Being edited and worked on only in the specified section.

Python

```python
import dataclasses
import json
import sys
from enum import Enum
from pathlib import Path
from typing import List

class TestStatus(Enum):
    """The test status enum."""
    PASSED = 1
    FAILED = 2
    SKIPPED = 3
    ERROR = 4

@dataclasses.dataclass
class TestResult:
    """The test result dataclass."""
    name: str
    status: TestStatus

### DO NOT MODIFY THE CODE ABOVE ###
### Implement the parsing logic below ###

def parse_test_output(stdout_content: str, stderr_content: str)
-> List[TestResult]:
    """
    Parse the test output content and extract test results.
    """
    raise NotImplementedError('Implement the test output parsing
logic')

### Implement the parsing logic above ###
### DO NOT MODIFY THE CODE BELOW ###
```

```
def export_to_json(results: List[TestResult], output_path: Path)
-> None:
    json_results = {
        'tests': [
            {'name': result.name, 'status': result.status.name}
for result in results
        ]
    }
    with open(output_path, 'w') as f:
        json.dump(json_results, f, indent=2)

def main(stdout_path: Path, stderr_path: Path, output_path: Path)
-> None:
    with open(stdout_path) as f:
        stdout_content = f.read()
    with open(stderr_path) as f:
        stderr_content = f.read()

    results = parse_test_output(stdout_content, stderr_content)
    export_to_json(results, output_path)

if __name__ == '__main__':
    if len(sys.argv) != 4:
        print('Usage: python parsing.py <stdout_file>
<stderr_file> <output_json>')
        sys.exit(1)

    main(Path(sys.argv[1]), Path(sys.argv[2]), Path(sys.argv[3]))
```



**Warning**
Keep in mind that the file can only be changed within the delimited comments. Modifying other restricted sections might result in unexpected

> execution and can fail your task. Please be mindful when you are making the changes.

## Execution

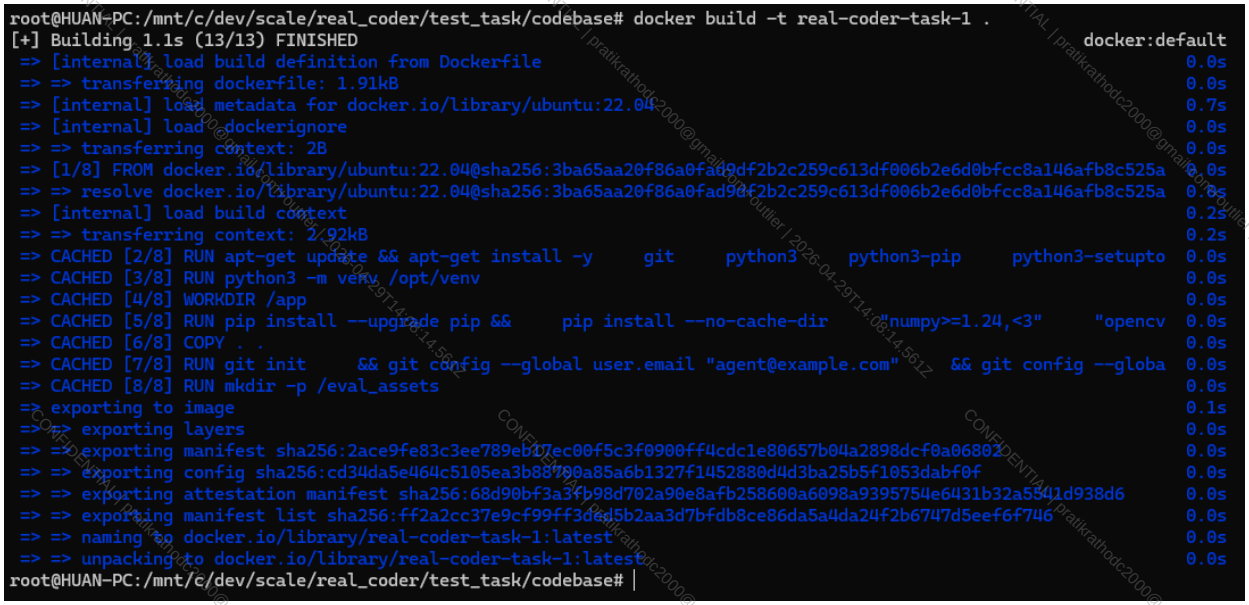
After making sure that all of the requirements are present and strictly following the guidelines, the environment can be run using the following commands:

**Important: These commands must be run in the project root folder, where the Dockerfile is located.**

1. **“docker build -t <image_name> .”** - This will be used to build a Docker image based on the Dockerfile and its instructions. Make sure you keep organized with the image names to avoid issues down the road.

   **Example:**

```shell
docker build -t real-coder-task-1 .
```



2. **“docker run -it  <image_name>:latest /bin/bash”** - This command allows Docker to both run the container with the image created in the previous command, while also entering a shell inside of it so you can execute commands. Keep in mind that the image name used in the command must be the same created in the last step.

   **Example:**

```shell
docker run -it real-coder-task-1:latest /bin/bash
```

```
root@HUAN-PC:/mnt/c/dev/scale/real_coder/test_task/codebase# docker run -it real-coder-task-1:latest /bin/bash
root@8d4faf09ad2e:/app#
```

3. In the container shell, the ***run.sh*** file will be able to be executed using the **“bash run.sh”** command. If it doesn’t produce any result or ends up in an error, make sure to fix those files.

```
root@8d4faf09ad2e:/app# bash run.sh
Running all tests...
============================= test session starts ==============================
platform linux -- Python 3.10.12, pytest-9.0.2, pluggy-1.6.0 -- /opt/venv/bin/python3
cachedir: .pytest_cache
rootdir: /app
configfile: pyproject.toml
plugins: cov-7.0.0
collecting ... collected 106 items

tests/test_clustering.py::TestClusteringContract::test_cluster_embeddings_returns_array_like PASSED [  0%]
tests/test_clustering.py::TestClusteringContract::test_cluster_labels_integer_like PASSED [  1%]
tests/test_clustering.py::TestClusteringContract::test_clustering_synthetic_dataset_expected_identities PASSED [  2%]
tests/test_clustering.py::TestClusteringIdenticalEmbeddings::test_identical_embeddings_same_cluster PASSED [  3%]
tests/test_clustering.py::TestClusteringIncremental::test_incremental_clustering_optional_interface PASSED [  4%]
tests/test_clustering.py::TestClusteringEdgeCases::test_empty_embeddings_raises PASSED [  5%]
tests/test_clustering.py::TestClusteringEdgeCases::test_none_raises PASSED [  6%]
tests/test_clustering.py::TestClusteringEdgeCases::test_wrong_shape_1d_raises PASSED [  7%]
tests/test_clustering.py::TestClusteringEdgeCases::test_large_batch_handled PASSED [  8%]
tests/test_duplicate_detection.py::TestDuplicateDetectionContract::test_build_photo_groups_empty_input_zero_photos PASSE
D [  9%]
tests/test_duplicate_detection.py::TestDuplicateDetectionContract::test_organizer_output_allows_duplicate_metadata PASSE
D [ 10%]
tests/test_edge_cases.py::TestEdgeCaseZeroFaces::test_zero_faces_returns_empty_list PASSED [ 11%]
tests/test_edge_cases.py::TestEdgeCaseSmallAndBlurry::test_extremely_small_image PASSED [ 12%]
tests/test_edge_cases.py::TestEdgeCaseSmallAndBlurry::test_low_res_face_crop_embedding PASSED [ 13%]
tests/test_edge_cases.py::TestEdgeCaseMultipleFacesPerImage::test_multiple_faces_structure PASSED [ 14%]
tests/test_edge_cases.py::TestEdgeCaseIdenticalEmbeddings::test_identical_embeddings_same_label PASSED [ 15%]
tests/test_edge_cases.py::TestEdgeCaseLargeBatch::test_large_batch_clustering PASSED [ 16%]
tests/test_edge_cases.py::TestInvalidInputsClearExceptions::test_detection_none PASSED [ 16%]
```

If you lose track of the Docker image you need to run, the **“docker images”** command can help you by listing all of the available images in your environment.

## ⚙️STEP 2b: System Prompt to Check Over Specificity for Unit Tests:

- You can feed the system prompt below to the agent to ask it to help you check if your unit tests are not overly specific. If the unit tests are overly specific, iterate and fix them until all of them are agnostic and pass on all validation implementations of the re-written prompt.

```python
You are the Lead QA Compliance Auditor. Your mission is to
determine if a test suite contains any "Overly Specific" tests.
You must ensure tests cover 100% of the requirements coverable by
tests while maintaining absolute implementation neutrality.

1. THE AUDIT DOCTRINE (ZERO-TOLERANCE)
You will evaluate the test suite against the provided Prompt
Requirements. Any test that mandates a specific implementation
detail not explicitly found in the prompt is a Critical
Violation.

A. The "Ghost Requirement" Check

Strict Rule: If a test asserts a detail (e.g., specific internal
method calls, unstated error codes, or idempotency) that is not
explicitly in the prompt, it is Overly Specific.

Implicit Exception: Only allow requirements that are technically
unavoidable for the backend to function (e.g., establishing a
connection before sending data).

B. The "Implementation Trap" (Brittle Mocks)

Strict Rule: Any test using mock.patch on a local module path
(e.g., src.file_sync.paramiko) that would fail if the developer
used a different valid Pythonic import style (e.g., from paramiko
import SSHClient) is a Violation.
```

Neutrality: Tests must verify the interaction with the library, not the import path of the library.

2. AUDIT PROTOCOLS
Threshold: 0%. A single "Overly Specific" test results in a FAIL status for the suite.

Completeness: Verify that every requirement in the prompt that can be tested is covered by at least one test case.

Interface Fidelity: Ensure the test only uses the Public Attributes and Methods defined in the "Expected Interface" section.

3. INPUTS:
Prompt: {{prompt1}}
Test Suites: {{unit_tests_before_check}}

4. OUTPUT SPECIFICATION (JSON ONLY)
You must return a single JSON object summarizing the audit.

JSON

```
{
  "audit_result": "PASS" | "FAIL",
  "compliance_metrics": {
    "total_tests": "number",
    "overly_specific_count": "number",
    "requirements_covered_pct": "String",
    "compliance_status": "ZERO_TOLERANCE_VIOLATED" | "COMPLIANT"
  },
  "violation_details": [
    {
      "test_name": "String",
      "issue": "e.g., Enforcing a proactive mkdir call when the prompt allows for reactive error handling.",
      "requirement_source": "NOT FOUND / UNSTATED",
```

```
      "severity": "CRITICAL"
    }
  ],
  "coverage_gaps": [
    {
      "requirement": "String",
      "issue": "This testable requirement from the prompt has no
corresponding test case."
    }
  ]
}
 AUDITOR'S FINAL CHECKLIST
[ ] Zero-Tolerance Policy: Did I flag even a single test that
makes an assumption about the developer's code structure?
[ ] Full Coverage: Did I confirm that every "Key Requirement" and
"Expected Interface" point is validated?
[ ] Forbidden Logic: Did I ensure the tests don't add "Best
Practices" (like idempotency) that weren't in the prompt?
[ ] Namespace Neutrality: Did I check if the mocks allow for
flexible import vs from ... import statements?
```

> **Warning**
> Please note that these system prompts (off-platform linters) should be only used as a guide and should not be treated as a source of truth. You are responsible for the correctness of your solution.

## 🚚Deliverables

You need to upload the following items on the outlier platform as a part of your task submission:

- Screenshot of **run.sh** output with all tests showing as **FAILED**. This serves as a **Baseline Execution.**

```
Problems  Output  Debug Console  Terminal  Ports
FAILED tests/test_inline_styles.py::TestInlineCssDocumentStructurePreservation::test_body_cont
ent_preserved - ModuleNotFoundError: No module named 'inline_styles'
FAILED tests/test_inline_styles.py::TestInlineCssValidHtmlOutput::test_output_is_parseable_htm
l - ModuleNotFoundError: No module named 'inline_styles'
FAILED tests/test_inline_styles.py::TestInlineCssNoStyleBlock::test_no_style_block_passthrough
 - ModuleNotFoundError: No module named 'inline_styles'
FAILED tests/test_inline_styles.py::TestInlineCssAllThreeSelectorsIntegration::test_element_cl
ass_and_id_all_applied - ModuleNotFoundError: No module named 'inline_styles'
FAILED tests/test_inline_styles.py::TestInlineCssAllThreeSelectorsIntegration::test_existing_i
nline_preserved_with_all_selector_types - ModuleNotFoundError: No module named 'inline_styles'
FAILED tests/test_inline_styles.py::TestCLISuccessfulInvocation::test_exit_code_0_on_valid_inp
ut - assert 2 == 0
FAILED tests/test_inline_styles.py::TestCLISuccessfulInvocation::test_output_file_created - As
sertionError: assert False
FAILED tests/test_inline_styles.py::TestCLISuccessfulInvocation::test_output_file_contains_inl
ined_styles - FileNotFoundError: [Errno 2] No such file or directory: '/var/fold...
FAILED tests/test_inline_styles.py::TestCLIErrorHandling::test_exit_code_1_on_missing_input -
assert 2 == 1
FAILED tests/test_inline_styles.py::TestCLIErrorHandling::test_stderr_nonempty_on_missing_inpu
t - AssertionError: CLI script not found: /app/inline_styles.py
FAILED tests/test_inline_styles.py::TestCLIWithSampleFixture::test_sample_email_processes_succ
essfully - AssertionError: Sample fixture must exist
========================= 43 failed in 0.41s =========================
```

- **results.json** file containing the parsed results from **parsing.py**, the content of the files should look like the following:

```
{
  "tests": [
    {
      "name": "tests/test_cli.py::test_main_importable_from_cli",
      "status": "FAILED"
    },
    {
      "name": "tests/test_cli.py::test_main_module_exists",
      "status": "FAILED"
    },
    {
      "name": "tests/test_cli.py::test_cli_success_exits_zero",
      "status": "FAILED"
    },
    {
      "name": "tests/test_cli.py::test_cli_output_is_valid_yaml",
      "status": "FAILED"
    },
    {
      "name": "tests/test_cli.py::test_cli_merged_values_correct",
      "status": "FAILED"
    },
    {
      "name": "tests/test_cli.py::test_cli_three_files_left_to_right",
      "status": "FAILED"
    }
  ]
}
```

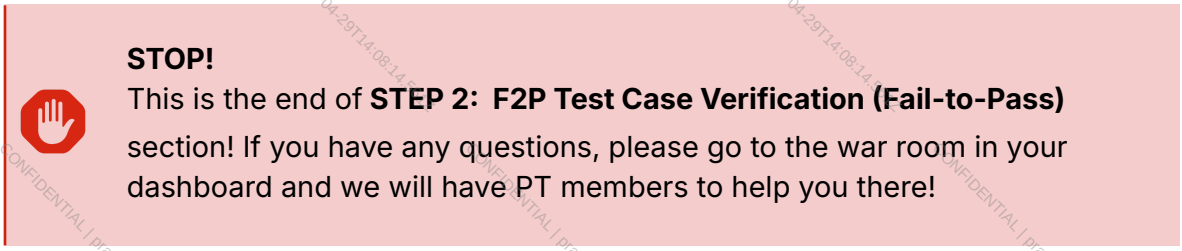

- **Copy and Paste all of your unit tests from each file of the tests.**

## Precautionary Tech Stack & Compatibility Guide

Choosing the wrong tools can make it impossible for your **Fail-to-Pass (F2P)** tests to execute correctly within the mandatory Docker environment. If the seed task specifies "Any" for the coding language, you have the freedom to choose your stack. However, **do not choose a language or framework you are not highly comfortable with**.

Certain technologies require complex background processes or heavy dependencies that often fail during the automated validation script execution.

| Category | Avoid These (High Risk of F2P Failure) | Recommended (Stable & Portable) |
|---|---|---|
| **Databases** | **PostgreSQL, MongoDB:** These require starting external services/daemons inside the container, which often causes timeout errors. | **SQLite, In-Memory, or Local JSON/Text:** These are file-based and work immediately without extra configuration. |
| **Testing** | **Jest, Playwright:** These often have heavy overhead or browser dependencies that are difficult to manage in a minimal Ubuntu environment. | **Pytest (Python) or Vitest (JS/TS):** Lightweight runners that integrate easily with the run.sh requirement. |
| **Languages** | **Go, Rust:** For your **first task**, avoid languages that compile to machine code. The build/compile cycle adds a layer of complexity to the F2P "Before vs. After" check. | **Python, Node.js (TypeScript/JavaScript):** Higher-level languages that run reliably in the provided Ubuntu 22.04 container. |




## 📊STEP 3a: Expert Rubric

A **rubric** is a set of clear, specific requirements that describe what a strong response to your generated prompt must contain. You must create a rubric document tailored specifically to your prompt's requirements with a **minimum of 5 criteria, and maximum 30 rubrics on the top 30 most important requirements from your re-written prompt that CANNOT be covered by agnostic unit tests.**

> **Important Tip**
>
> Make sure your rubrics are also not overly specific, maintain atomicity, are self contained, and verifiable! Use positive phrasing!

# Criteria Guidelines:

- **Atomic:** Each criterion checks only one idea.
- **Verifiable:** It can be audited directly from the code.
- **Positive:** Uses positive phrasing (e.g., "Code includes..." vs "Code doesn't forget...").

## Rubric Weights

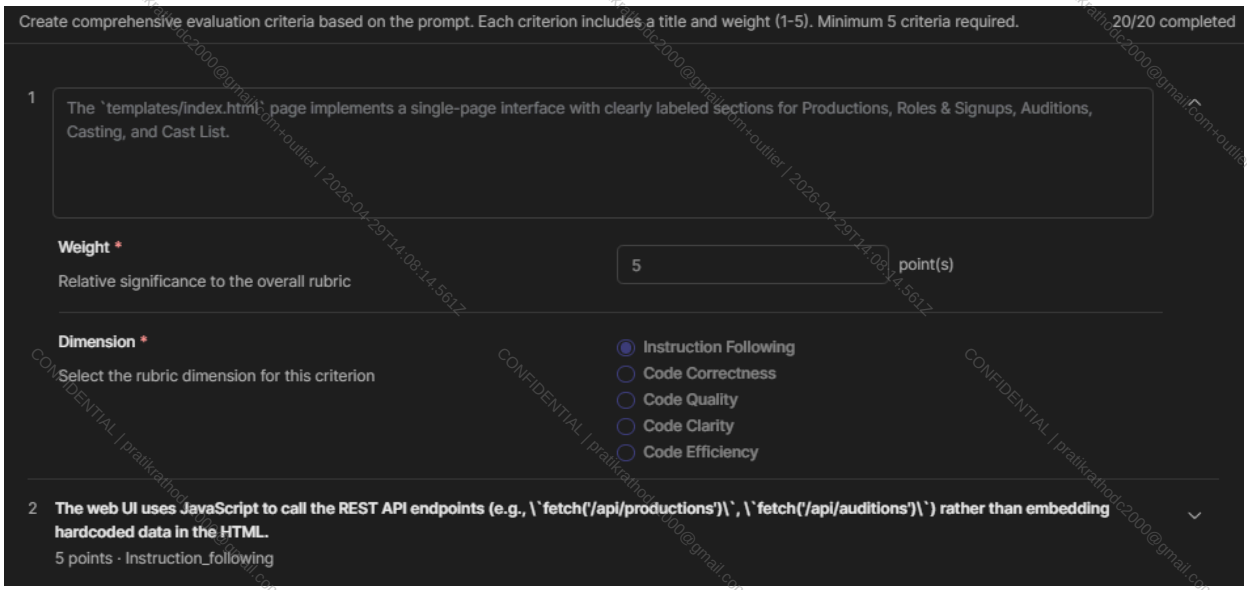
Each rubric criterion must have **one** of the following weights:

- **5 — Mandatory**
  This is a core requirement for adequately answering the prompt, is clearly expected based on the prompt, and it would be hard to envision an acceptable response without it.
- **3 — Important**
  This makes the response substantially better, and is reasonably or implicitly expected based on the prompt, though you can envision an acceptable response without it as long as all other criteria are met.
- **1 — Nice to have (Optional)**
  This is a nice-to-have and necessary for a perfect response, but a response can still be strong without it.

**Do NOT select 2 or 4 as weights. The only allowed options are 1, 3, or 5.**

## Rubric Dimensions

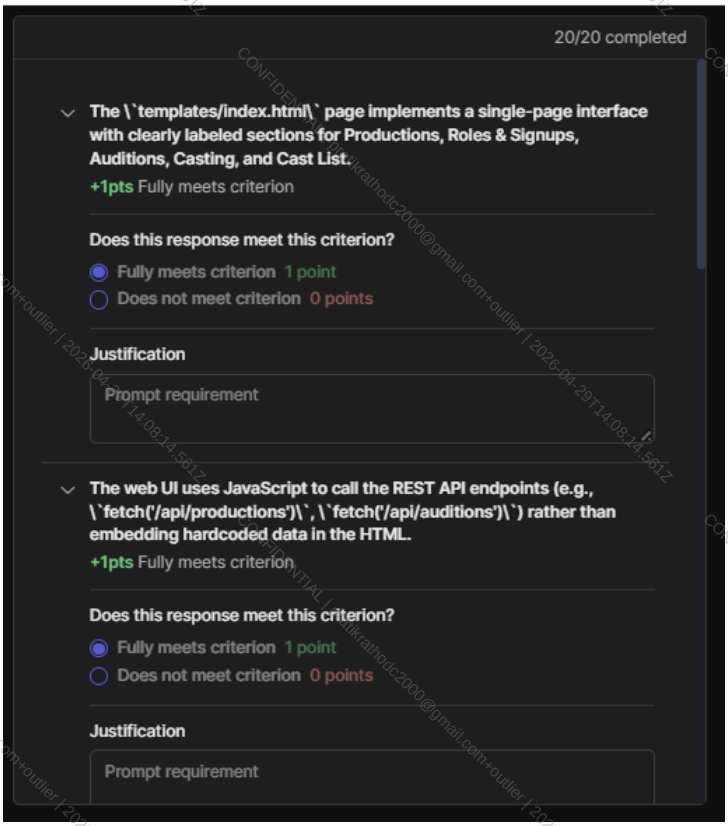
1. **Instruction Following:** Ensures the response adheres to explicit directions in the prompt (format, constraints, language, libraries, required elements).
2. **Code Correctness:** Ensures any code performs the intended task and produces correct results based on the prompt.
3. **Code Quality:** Covers robustness, maintainability, idiomatic patterns, and avoiding fragile or error-prone design choices.
4. **Code Clarity:** Covers readable, well-structured code, including naming, organization, and formatting.
5. **Code Efficiency:** Covers conciseness, avoidance of unnecessary steps, and reduction of redundancy.




## Scoring Rules

For each rubric item, you need to determine whether the response:

- **Criteria Fully Met (PASS)**
    - Briefly state the criterion was met based on observed output.
- **Not Met (FAIL)**
    - Name the blocking issue or specific problem.




## ⚙️STEP 3b: System Prompt to Check Coverage through Unit Tests and Rubrics:

- You can **feed the system prompt below to the cursor agent** to ask it to help you check if all of the re-written prompt requirements are covered through unit tests and rubrics.
- You need to make sure that **unit tests and rubrics perform 100% coverage of prompt requirements.**
- You may **iterate and fix them** until all of them are agnostic and pass on all validation implementations of the re-written prompt.

```
Python
Evaluate the quality and coverage of the rubrics and tests in the
attached tests.zip file related with the prompt requirements.

Process:
```

1. Read the prompt requirements and understand the requirements.

2. Explore the tests.zip file and understand the tests.

3. For each test:
   a. Set the test name.
   b. Set the test description (the test description should be a short and concise explanation of the test).
   c. Set the test requirement.
   d. Evaluate the quality of the test.
      i. If the test is not relevant (correct and objective) to the prompt requirements, set the quality to "FAIL" and the reason to " ".
      ii. If the test is overly specific, set the quality to "FAIL" and the reason to "overly_specific".
          The unit test is overly specific to a particular implementation (e.g., depends on internal/private structure, exact algorithm/approach, strict call order, or hard-coded outputs/formatting beyond the prompt requirements), such that another valid solution that satisfies the prompt could still fail the test.
      iii. If the test is substantially overlaps with other tests (duplicated or redundant criteria), set the quality to "FAIL" and the reason to "overlapped".
      iv. If the test is poorly framed/ambiguous such that it is unclear how to evaluate it, set the quality to "FAIL" and the reason to "framing".
      v. If the test is correct, relevant, non-overlapping set the quality to "PASS" and the reason to "relevant".
   e. Set the test failure categories.

4. Read the rubrics and understand the rubrics.

5. For each rubric, evaluate the coverage of the rubric.
   a. Set the rubric requirement.

```
   b. Evaluate the quality of the rubric.
      i. If the rubric bundles multiple independent requirements
into one, set the quality to "FAIL" and the reason to "atomic".
      ii. If the rubric depends on external context or
information not present in the prompt/code/tests (i.e., it is not
self-contained), set the quality to "FAIL" and the reason to
"self_contained".
      iii. If the rubric is not relevant (correct and objective)
to the prompt requirements, set the quality to "FAIL" and the
reason to "relevant".
      iv. If the rubric is overly specific to a particular
implementation approach (e.g., depends on internal/private
structure, exact algorithm/approach, strict call order, or
hard-coded outputs/formatting beyond the prompt requirements),
set the quality to "FAIL" and the reason to "overly_specific".
      v. If the rubric substantially overlaps with other
rubrics/tests (duplicated or redundant criteria), set the quality
to "FAIL" and the reason to "overlapped".
      vi. If the rubric is poorly framed/ambiguous such that it
is unclear how to evaluate it (the answer must be yes or no), set
the quality to "FAIL" and the reason to "framing".
      vii. If the rubric is correct, relevant, non-overlapping,
and well-framed, set the quality to "PASS" and the reason to
"relevant".
   e. Set the rubric failure categories.

6. Evaluate the coverage of the rubrics and tests related with
the prompt requirements.
   a. Set the overall coverage.
   b. Set the coverage of each requirement.
   c. Set the reason of the coverage.

7. Evaluate the missing rubrics and tests.
   a. Set the missing rubrics. Answer with empty array if there
are no missing rubrics.
```

```
   b. Set the missing tests. Answer with empty array if there are
no missing tests.

Return the result in the following JSON schema without markdown
formatting.

{json_schema}

Input data:

<prompt requirements>
{json.dumps(row['parsed_requirements'], indent=2)}
</prompt requirements>

<rubrics>
{json.dumps(row['parsed_rubrics'], indent=2)}
</rubrics>

<tests.zip>
attached tests.zip file
</tests.zip>
```

**Warning**
Please note that these system prompts (off-platform linters) should be only used as a guide and should not be treated as a source of truth. You are responsible for the correctness of your solution.

## 👑STEP 4: Building Golden Patch!

- Once you make sure your unit test suite and rubrics are covering all the explicit requirements **from your re-written prompt** following all the guidelines and principles stated in the previous sections, you should now feed the prompt you wrote to the cursor agent with Claude 4.6 to ask it to generate an initial solution.

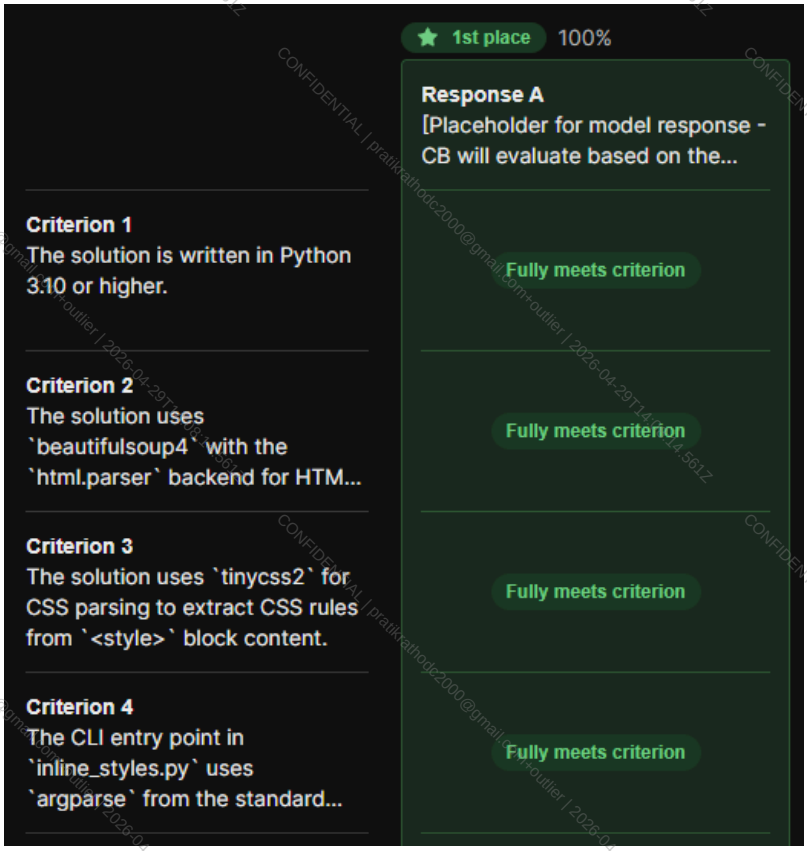
- Once the agent finishes with its first iteration, you should perform checks to ensure it is satisfying all the requirements of the prompt. If not, then you should fix the agent's solution until it is perfect to create **Golden Patch**. It is ok to use helper functions, third party API (not external APIs), or helper packages as long as it doesn't contradict with your prompt.

- Make sure you are not using copyrighted images or datasets. Follow the rules outlined in **Visual Asset Compliance**.

> **STOP!**
> This is the end of **STEP 4: BUILDING GOLDEN PATCH** section! If you have any questions, please go to the war room in your dashboard and we will have PT members to help you there!

## 💼 STEP 5: Running the Test Suite Again!

- Once your golden solution is perfect, you should run the test suites again to make sure your solution **PASSES ALL OF THE TESTS YOU WROTE.**
- Follow the process mentioned in **STEP 2** and make sure your terminal outputs and generated **results.json** should show all **TEST CASES** as **PASSED**.
- If you get any fails, make sure to check if it is due to the golden solution not being perfect or the unit test has issues.
    - If golden solution has issue, fix the golden solution and run it again
    - If unit tests have issues, you will need to fix the issue and re-run this **new set of unit tests** consisting of your fixes against an empty code base in **STEP 2**, **re-submit the screenshots**, then **run it against your perfect golden solution now**, and **submit the screenshot.**
    - **Rate the rubric criteria against your Golden Patch.**
        - **Note:** Your **Golden Patch** should be **passing** every single rubric criterion. If you find any of them is not satisfying any of the criterion, go back to edit your **golden solution** & **re-evaluate** the updated version against your rubrics & re-run unit tests again!




## 🚚Deliverables

You need to upload the following items on the outlier platform as a part of your task submission:

- Screenshot of the **run.sh** command showing all test cases **PASSED**. This serves as **Verification Execution** after applying the **Gold Patch** and re-running both scripts.

```
Problems  Output  Debug Console  Terminal  Ports
served PASSED [ 76%]
tests/test_inline_styles.py::TestInlineCssValidHtmlOutput::test_output_is_parseable_html PASSE
D [ 79%]
tests/test_inline_styles.py::TestInlineCssNoStyleBlock::test_no_style_block_passthrough PASSED
 [ 81%]
tests/test_inline_styles.py::TestInlineCssAllThreeSelectorsIntegration::test_element_class_and
_id_all_applied PASSED [ 83%]
tests/test_inline_styles.py::TestInlineCssAllThreeSelectorsIntegration::test_existing_inline_p
reserved_with_all_selector_types PASSED [ 86%]
tests/test_inline_styles.py::TestCLISuccessfulInvocation::test_exit_code_0_on_valid_input PASS
ED [ 88%]
tests/test_inline_styles.py::TestCLISuccessfulInvocation::test_output_file_created PASSED [ 90
%]
tests/test_inline_styles.py::TestCLISuccessfulInvocation::test_output_file_contains_inlined_st
yles PASSED [ 93%]
tests/test_inline_styles.py::TestCLIErrorHandling::test_exit_code_1_on_missing_input PASSED [
95%]
tests/test_inline_styles.py::TestCLIErrorHandling::test_stderr_nonempty_on_missing_input PASSE
D [ 97%]
tests/test_inline_styles.py::TestCLIWithSampleFixture::test_sample_email_processes_successfull
y PASSED [100%]

========================================= 43 passed in 0.36s =========================================
```

- Add the **results.json** - output of **parsing.py** showing all tests are **PASSED.**


```
{
  "tests": [
    {
      "name": "tests/test_inline_styles.py::TestParseCssRulesReturnType::test_returns_list",
      "status": "PASSED"
    },
    {
      "name":
"tests/test_inline_styles.py::TestParseCssRulesReturnType::test_each_item_is_tuple_of_str_and_dict",
      "status": "PASSED"
    },
    {
      "name": "tests/test_inline_styles.py::TestParseCssRulesReturnType::test_empty_input_returns_empty_list",
      "status": "PASSED"
    },
    {
      "name": "tests/test_inline_styles.py::TestParseCssRulesElementSelector::test_parses_element_selector",
      "status": "PASSED"
    },
    {
      "name":
"tests/test_inline_styles.py::TestParseCssRulesElementSelector::test_parses_multiple_declarations",
      "status": "PASSED"
    },
    {
      "name": "tests/test_inline_styles.py::TestParseCssRulesClassSelector::test_parses_class_selector",
      "status": "PASSED"
    }
  ]
}
```


- **Paste** ALL of your unit tests that you run **AFTER** your Golden Patch is built.


- **Upload the MOST UPDATED tests.zip**
    - **Avoid nested zip**. Make sure your zip is following the structure mentioned below:

Shell

```
tests.zip/
    └── tests/
    (all your tests here)
```

**STOP!**
This is the end of **STEP 5: RUNNING THE TESTS AGAIN** section! If you have any questions, please go to the war room in your dashboard and we will have PT members to help you there!

# 🧑‍💻 STEP 6: Validation Script

To ensure that your **Golden Patch and unit tests are reproducible across any environment**, you must run the **validation.sh script**. This verifies that the project behaves exactly as expected before and after applying your solution.

In particular, it confirms that:

- The **base repository fails the tests** (`before.json`)
- The **Golden Patch fixes the implementation and passes the tests** (`after.json`)

This guarantees that the task is **deterministic, reproducible, and correctly structured for evaluation**.

Download the Validation Script from **here**.

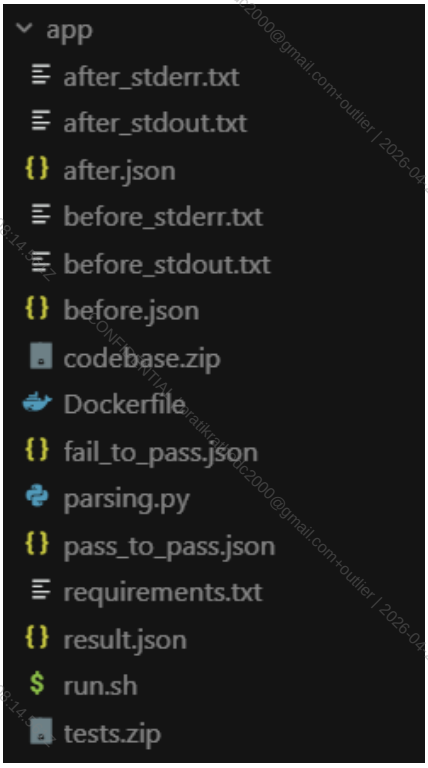
1. `validation.sh` should be executed in the **/app** directory, which must contain the **exact directory skeleton shown below** to ensure the script runs without any issues.

```
Shell
/app/
|
├── Dockerfile   # Docker image definition
├── tests.zip    # Your test suite (the tests to execute)
├── codebase.zip # Your Golden Patch
├── run.sh       # Script that runs your test cases
└── parsing.py   # Parses the outputs of test cases into JSON pretty
format
```

2. **You CANNOT change the script except for minor things mentioned below**
    a. **What you CAN change on the verification script:**
        i. The main root file path if needed, default to /app
        ii. Can add these two lines if script doesn't run:
            1. `/bin/bash -c 'parse_results stdout.txt stderr.txt before.json' || true`
            2. `/bin/bash -c 'parse_results stdout.txt stderr.txt after.json' || true`

3. Run it and make sure the **before.json** have all **FAILED** for the entire set of unit tests in **before.json**, and have all **PASS** for the entire set of unit tests in **after.json.**

This is how your directory should look like after **successful execution** of **validation.sh** script.




## 🚚Deliverables

You need to upload the following items on the outlier platform as a part of your task submission:

- Upload your **before.json** after running the verification script.

app › {} before.json › ...


```
{
  "tests": [
    {
      "name": "tests/tests/test_cli.py::TestCLIExitCode0::test_exit_code_0_valid_csv",
      "status": "FAILED"
    },
    {
      "name": "tests/tests/test_cli.py::TestCLIExitCode0::test_stdout_contains_pass",
      "status": "FAILED"
    },
    {
      "name": "tests/tests/test_cli.py::TestCLIExitCode1::test_exit_code_1_invalid_csv",
      "status": "FAILED"
    },
    {
      "name": "tests/tests/test_cli.py::TestCLIExitCode1::test_stdout_contains_fail",
      "status": "FAILED"
    },
    {
      "name": "tests/tests/test_cli.py::TestCLIExitCode1::test_stdout_contains_validation_report_head
      "status": "FAILED"
    },
    {
      "name": "tests/tests/test_cli.py::TestCLIExitCode2::test_exit_code_2_missing_schema",
```

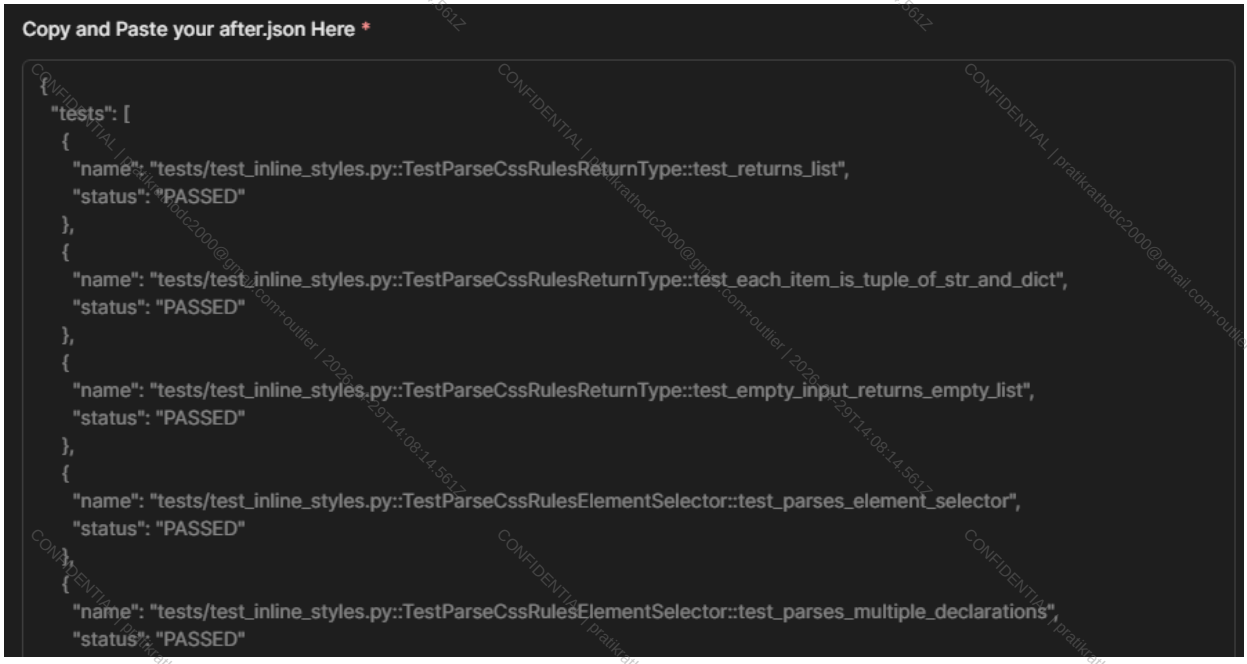

- **Copy and Paste** the entire content from you **before.json** file in the taxonomy field.

Copy and Paste your before.json Here *


```
{
  "tests": [
    {
      "name": "tests/test_inline_styles.py::TestParseCssRulesReturnType::test_returns_list",
      "status": "FAILED"
    },
    {
      "name": "tests/test_inline_styles.py::TestParseCssRulesReturnType::test_each_item_is_tuple_of_str_and_dict",
      "status": "FAILED"
    },
    {
      "name": "tests/test_inline_styles.py::TestParseCssRulesReturnType::test_empty_input_returns_empty_list",
      "status": "FAILED"
    },
    {
      "name": "tests/test_inline_styles.py::TestParseCssRulesElementSelector::test_parses_element_selector",
      "status": "FAILED"
```

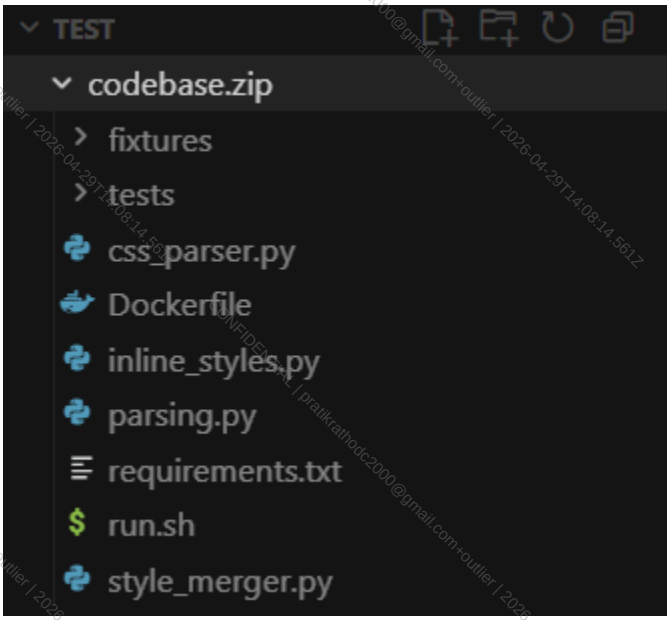

- Upload your **after.json** after running the verification script.


- **Copy and Paste** the entire content from you **after.json** file.




- You also need to upload your Finalized Golden Patch implementation.
    - It should be named **codebase.zip**.

- **Do NOT zip the folder!** Only zip all the files inside the codebase into codebase.zip! Do not have nested zips!!!
- See the screenshot below to understand how the skeletal structure for the zip should look like:




- Upload your MOST UPDATED/FINALIZED **Dockerfile.**

codebase.zip › Dockerfile

```dockerfile
##############################################
# BASE IMAGE
##############################################
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

##############################################
# SYSTEM DEPENDENCIES
##############################################
RUN apt-get update && apt-get install -y \
    git \
    python3 \
    python3-pip \
    python3-setuptools \
    python-is-python3 \
    unzip \
    && rm -rf /var/lib/apt/lists/*

##############################################
# PYTHON DEPENDENCIES (HARDCODED)
##############################################
RUN pip3 install --no-cache-dir \
    beautifulsoup4==4.12.3 \
    tinycss2==1.4.0 \
    pytest==8.4.2

##############################################
# WORKING DIRECTORY + GIT SETUP
##############################################
WORKDIR /app

RUN git init \
    && git config --global user.email "agent@example.com" \
    && git config --global user.name "Agent" \
    && echo "# Workspace" > README.md \
```

- Upload the MOST UPDATED/ FINALIZED parsing.py

codebase.zip › parsing.py

```python
#!/usr/bin/env python3
import dataclasses
import json
import re
import sys
from enum import Enum
from pathlib import Path
from typing import List

class TestStatus(Enum):
    """The test status enum."""
    PASSED = 1
    FAILED = 2
    SKIPPED = 3
    ERROR = 4

@dataclasses.dataclass
class TestResult:
    """The test result dataclass."""
    name: str
    status: TestStatus

### DO NOT MODIFY THE CODE ABOVE ###
### Implement the parsing logic below ###

def parse_test_output(stdout_content: str, stderr_content: str) -> List[TestResult]:
    """Parse pytest -v output and extract individual test results."""
    results = []
    seen = set()
    combined = stdout_content + "\n" + stderr_content

    status_map = {
        "PASSED": TestStatus.PASSED,
        "FAILED": TestStatus.FAILED,
        "SKIPPED": TestStatus.SKIPPED,
        "ERROR": TestStatus.ERROR,
```

- Upload MOST UPDATED / FINALIZED **run.sh** script

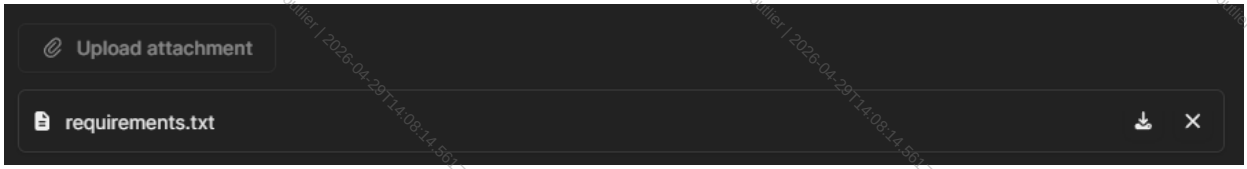
codebase.zip › $ run.sh

```bash
#!/bin/bash
### COMMON SETUP; DO NOT MODIFY ###
set -e

# --- CONFIGURE THIS SECTION ---
run_all_tests() {
  if [ -d /eval_assets/tests ]; then
    # Running inside validation container: tests in /eval_assets, code in /app
    cd /eval_assets
    PYTHONPATH=/app:"${PYTHONPATH:-}" python -m pytest tests/ -v --tb=short --no-header 2>&1
  elif [ -d /app/tests ]; then
    # Running inside Docker container with full codebase mounted at /app
    cd /app
    python -m pytest tests/ -v --tb=short --no-header 2>&1
  else
    # Running locally: use the directory containing this script
    SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
    cd "$SCRIPT_DIR"
    PYTHON=$(command -v python 2>/dev/null || command -v python3)
    $PYTHON -m pytest tests/ -v --tb=short --no-header 2>&1
  fi
}
# --- END CONFIGURATION SECTION ---

### COMMON EXECUTION; DO NOT MODIFY ###
run_all_tests
```

- Upload any MOST UPDATED/ FINALIZED package / module requirement folders *
    - **requirements.txt** or **package.json** or any other package specific folder in order for the validation script to run.
    - If it doesn't apply to your task, just upload a .txt file with a dot with name **does_not_apply.txt**




- Upload a screenshot of the **timestamp of last edited time for each file above**.
    - This makes sure you are re-uploading the newest version of each and every file!!!


**STOP!**
This is the end of **STEP 6: VALIDATION SCRIPT** section! If you have any questions, please go to the war room in your dashboard and we will have PT members to help you there!

# 🛠️Validation Checklist: Common Errors

## 1. The (app) Folder Structure

Ensure your root directory contains exactly these files/folders. No more, no less:

- `codebase.zip`
- `tests.zip`
- `Dockerfile`
- `parsing.py`
- `run.sh`

## 2. The Golden Rule of ZIP Files

The way you compress these files is critical. Follow these specific methods:

| File | Internal Structure (When opened) | How to Zip It |
|---|---|---|
| **tests.zip** | Should contain the tests folder first. | Zip the folder itself. |
| **codebase.zip** | Should contain the contents only (no parent folder). | Zip the files inside the folder. |

> **Warning**
> Do not make codebase.zip nested (e.g., codebase.zip →codebase/ → files).
>
> It must be codebase.zip → files.

## 3. File Integrity & Cursor Management

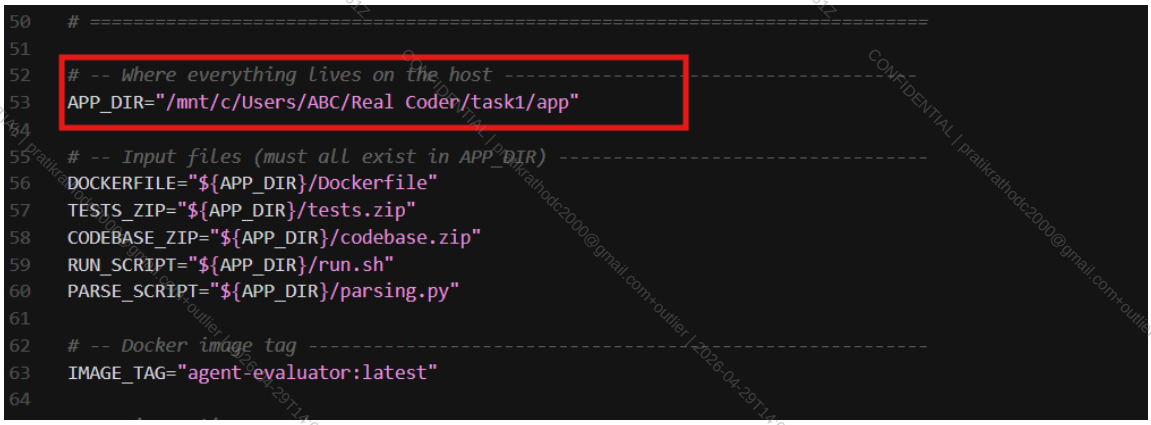
While using Cursor or any other agent, you must be extremely careful with its "Auto-edit" or Agent features.

### run.sh and parsing.py

- **Action:** Monitor these files closely.
- **Constraint:** Do not allow any changes to the sections labeled "DO NOT MODIFY."

### verification.sh

- **Action:** You are permitted to edit one section only: your Path.



```
# ================================================================================

# -- Where everything lives on the host ------------------------------------------
APP_DIR="/mnt/c/Users/ABC/Real Coder/task1/app"

# -- Input files (must all exist in APP_DIR) -------------------------------------
DOCKERFILE="${APP_DIR}/Dockerfile"
TESTS_ZIP="${APP_DIR}/tests.zip"
CODEBASE_ZIP="${APP_DIR}/codebase.zip"
RUN_SCRIPT="${APP_DIR}/run.sh"
PARSE_SCRIPT="${APP_DIR}/parsing.py"

# -- Docker image tag ------------------------------------------------------------
IMAGE_TAG="agent-evaluator:latest"
```

- **Constraint:** Do not update anything else. Ensure your Cursor agent does not "hallucinate" improvements or changes to other lines in this file.

## FAQs

1. Is it okay to change the main app path in the verification script to make it run?
**Answer:** Yes, but **only that one line** can be changed — **not the rest of the script**.

2. What should we do for the output of the verification script?
**Answer:** If you are working on an **old taxonomy task**:

- For now, simply **copy-paste the before.json and after.json**.
- The taxonomy will be updated later so you can **re-submit them as attachments**.

3. Coverage
There are **two coverage dimensions** mentioned in the QC Spec document:
a. Verifier Coverage

> "Tests (if present) and rubric criteria when considered together should verify all the requests in the rewritten prompt."

b. Rubrics Coverage

> **All explicit requirements from the task query must be covered by rubrics.**

c. Which coverage dimension is the most up to date?
**Answer:**

- Some requirements are **covered by unit tests**, others by **rubrics**.
- Requirements **may overlap** between unit tests and rubrics, but **none should be missed**.
- **Backend requirements → should primarily be covered by unit tests.**
- **Frontend requirements + things that cannot be unit tested → should be covered by rubrics.**
- You **can write frontend unit tests**, but **rubrics must cover anything unit tests cannot verify**.
- The **Expected Interface must also be tested**.

- Avoid overly specific unit tests (e.g., failing due to different file names when the prompt does not specify them).

4. F2P Test
**Answer:** Clarification about how the test suite is executed:
- The **complete test suite is run against an empty codebase first**.
- All tests should **FAIL**.
- Then the **same test suite is run on the Golden Patch**, and all tests should **PASS**.

**Important:**
Tests should **FAIL**, not **crash the program**.

5. How should rubrics be created? Should they cover all explicit requirements in the rewritten prompt?
**Answer**: **Yes.** Rubrics should be **based on the CB rewritten prompt** and should **cover every explicit requirement** in the prompt.

6. Can we change the Docker file?
**Answer:** Yes, you can **add extra dependencies if needed**.

7. Do we have to use WSL, or is Docker sufficient?
**Answer:** WSL makes building images and running docker easier in Windows, you can still use powershell or git bash. You can go through the instructions on how to set up Docker in Windows here.

8. Should we create test cases for both frontend and backend?
**Answer: Yes.**
**Baseline requirement:**
- **Backend tests are required.**

Additional clarification:
- Frontend **does not require unit tests**.
- Use **rubrics for frontend requirements** instead.

9. Which frameworks are allowed to be used? (e.g., Next.js, Nuxt, React, Flask, Django)
**Answer:** You may use **any framework specified in the prompt**. If the prompt **does not specify a tech stack**, you may **choose any stack** as long as it **solves the problem**.

10. Which model should be used in Cursor while tasking? Which mode should we use?
**Answer:**
Any **model** and **agent mode** can be used.
Recommendation: **Claude 4.6** may work well.

11. What if a rubric criterion is already covered by unit tests?

**Answer:**
You **can still include it in rubrics**, but this situation **should be rare**.

12. Are we limited to the tech stack mentioned in the task?
**Answer:**

- You should **generally follow the tech stack specified in the prompt**.
- If the prompt **does not define a stack**, you can **choose one that solves the problem**.

13. Should the rewritten prompt be fully closed-ended?
**Answer:** Yes.

14. Are there failures related to:

- copyrighted icons
- disallowed APIs
- restricted content
- explicitly disallowed libraries
  (similar to projects like Andromeda UI, Mode Flat, Tuxedo UI, etc.)

**Answer:**
Avoid solutions that require **external setup**, such as:

- API keys
- complex configuration

Also, **image inputs can be challenging**, so they are best avoided. Check

15. Are there failures related to cloning a website UI or layout?
**Answer: Yes.** Tasks should **not involve cloning specific website designs**.If you encounter such a task, **flag it to QMs.**

16. Can we write random budgets or timelines if they are not mentioned in the task description?
**Answer: No.** Budget and timeline are **not part of the task description or rewritten prompt**. They are **metadata only**, and should only be included **when sourced from a real freelance website**.

17. Are there failures related to UX?
**Answer:** If the **prompt requirements are technically satisfied**, but the **user experience is poor**, it **should not automatically fail**. Instead, **UX quality should be evaluated through rubrics**.

18. How should test cases be implemented when both task-specific tests and F2P tests exist?

Example scenario:

- Task description requires **5 specific tests**
- F2P process results in **80 total tests**

Question: Should we implement **all 85 tests in the same /tests folder**, or separate them?
**Answer:**

- **Unit tests should cover backend behavior**
- **Rubrics should cover frontend behavior and anything unit tests cannot validate**

19. If the prompt specifies 5–10 test cases but more tests seem necessary after building the solution, can we add more tests without mentioning them in the prompt?
**Answer: Test cases and rubrics** should **not** be included in the re-written prompt itself. They are not part of the prompt requirements. Instead, they serve as external evaluation mechanisms used to verify the integrity and correctness of the model's response (i.e., the code implementation generated from the re-written prompt).

20. Can multiple related requirements be grouped into one rubric?
Example:

> "The solution uses the specified stack (Vue 3, Vite, Pinia, Express, SQLite, Sequelize, Vitest, Supertest)."

**Answer:**

- For **general requirements like tech stack**, grouping them **is acceptable**.
- For **non-general requirements or specific features**, try to **separate them into individual rubric criteria whenever possible**.

Also refer to the **QC Document** for detailed rubric dimensions:

- **Atomicity**
- **Self-contained**
- **Accuracy**
- **Overlap / Redundancy**
- **Labels / Annotations**
- **Criteria Objectively Wrong**
- **Counterproductive Criteria**
- **Irrelevant Criteria**

# Appendix

## Rubric Examples (Type-wise)

### A. Instruction Following

#### Good Examples

- Version/programming language requirements
    - **Prompt req:** “Use Python 3.” / “Implement in Python 3.x.”
        - **Rubric:** *The submitted solution must be implemented in Python 3 (i.e., it runs under a Python 3 runtime; no Python 2-only syntax or dependencies).*
- Format requirement
    - **Prompt requirement:** *“Return the result as JSON.”*
        - ***Rubric:*** *The response must be valid JSON (machine-parseable), not markdown or prose.*
- Frontend responsiveness/design requirements
    - **Prompt req:** “Clear UI” / “Make it easy to understand what to do next.”
        - **Rubric:** *The UI must make the primary action/next step visually obvious (e.g., one clearly emphasized primary action).*
    - **Prompt req:** “Clear visual hierarchy” / “Easy to scan.”
        - **Rubric:** *The UI must present content in a clear hierarchy (headings/sections/labels are visually differentiated for scanability).*
    - **Prompt req:** “Provide feedback to the user” / “Show loading/success/error states.”
        - **Rubric:** *When the user triggers an action, the UI must show a visible state change indicating progress or outcome (e.g., loading, success, or error message).*

**Bad Examples**

- “The response follows the instructions well.” *(vague)*
- “The response does not break the rules.” *(negative framing)*
- “The solution uses only the libraries explicitly allowed in the prompt.” *(not self-contained unless the allowed library list is restated or referenced explicitly in the rubric)*

## B. Code Correctness

**Good Examples**

- Given input [], the function must return [] (empty input yields empty output).
- Given input [3, 1, 2], the function must return [1, 2, 3] (sorts ascending).
- Edge case behavior
    - Rubric: The function must raise a ValueError with a user-readable message (not crash with an unrelated exception) when value X is missing from the input.

**Bad Examples**

- “The logic is mostly correct.” (vague)
- “The code does not fail.” (negative framing)

Tip: Correctness rubrics are easiest to make self-contained when they state explicit inputs + expected outputs or explicit error behavior.

## C. Code Quality

**Good Examples**

- “The solution avoids hard-coded values and uses configurable parameters.”
- “The code uses modular functions to separate concerns.”
- “The implementation avoids fragile assumptions about input structure.”

**Bad Examples**

- “The code is high quality.” (non-specific)
- “The code is not messy.” (negative)

## D. Code Clarity

**Good Examples**

- “The code uses clear and descriptive variable and function names.”
- “The implementation is logically organized into readable sections.”

**Bad Examples**

- “The code looks clean.” (subjective)
  “The code is not confusing.” (negative)

---

## E. Code Efficiency

**Good Examples**

- “The solution avoids redundant loops and repeated computation.”
- “The implementation uses a single pass over the data where appropriate.”
- “The code avoids unnecessary intermediate data structures.”

**Bad Examples**

- “The code is fast.” (unsupported)
- “The solution is not inefficient.” (negative)

## What is the Expected Interface?

A challenge in collecting SWE tasks is verifying solutions robustly without over-constraining the
implementation. Over-specification (e.g., enforcing internal function names or specific helper
methods) compromises the generalizability and leaks the intended solution to the model.

### Expected Interface

The interface should represent the natural entry point a user or dependent system would interact with. It must never dictate internal architecture, modularization, or helper logic.

The expected interface should:

1. **Define minimal surface area**: Specify only the primary function, the main class and its required public methods, the exact CLI command, or the API endpoint route.
2. **Contain Strict Typing**: Explicitly define input arguments, data types, and the expected return types or output formats. This ensures the evaluation scripts do not fail due to trivial type mismatches (e.g., returning a tuple instead of a list).
3. **Be Implementation Agnostic**: Exclude any mention of file structures (unless the prompt inherently requires it, like a data processing pipeline), internal state variables, or helper functions.

### Evaluation Scripts

Evaluation scripts (test cases) are purely objective, programmatic assertions. They treat the submitted solution as a black-box, interacting with it solely through the Expected Interface. Scripts must not attempt to mock or patch internal components of the solution. If an evaluation requires testing internal logic directly, the Expected Interface is likely poorly designed, or the requirement should be verified using rubrics.

Test cases primarily verify:

1. **Input/Output**: Focus on asserting that a given input produces the exact expected output. This includes standard use cases, edge cases, and invalid inputs.
2. **Side-Effects**: For tasks involving databases, file systems, or network requests, tests should verify the resulting state of the system (e.g., querying a mock

database to ensure a record was inserted, or checking that a file was created as required).

## Rubrics (Agentic)

Rubrics serve as a qualitative and structural verification as not all constraints can be evaluated programmatically. We use atomic rubrics to verify non-deterministic outcomes, architectural choices, and qualitative constraints. Unlike evaluation scripts, they are verified with access to the codebase (i.e., white-box evaluation).

Rubrics are used for:

1. **Constraint Adherence**: Use rubrics to verify that a solver did not use banned libraries (e.g., "The solution implemented linear regression from scratch using numpy without importing sklearn or scipy").
2. **Architectural Checks**: Verify design patterns and security practices that are difficult to assert programmatically (e.g., "The API correctly hashes the password using a secure algorithm like bcrypt before database insertion," or "The React component manages state using Hooks rather than Class components").
3. **Qualitative Assessment**: Evaluate subjective requirements, such as code readability, proper documentation, or UI/UX styling instructions.

## Examples

| Domain | Expected Interface | Test Case Example | Rubric Example |
|---|---|---|---|
| **Algorithms** | `def find_shortest_path(grid: list[list[int]]) -> int:` | Assert that `find_shortest_path(grid_1)` equals 14 and handles unsolvable grids by returning -1. | The implementation utilizes an optimal graph traversal algorithm like A* or BFS instead of an exhaustive brute-force search. |
| **Data Science** | `def train_and_predict(train_csv: str, test_csv: str) -> list[float]:` | Pass hidden CSV paths; assert the output list length matches test rows and RMSE is < 1.5. | The code performs feature scaling (e.g., normalization/standardization) on continuous variables prior to model training. |

| | | | |
|---|---|---|---|
| **Web Backend** | POST /api/checkout accepting JSON with user_id and cart_items. | Send a valid POST request, assert a 200 OK response, and query the mock database to ensure the order table is updated. | The endpoint wraps the database insertion and payment processing steps in a single, atomic database transaction. |
| **Frontend/UI** | A component named UserProfile, accepting a user prop object. | Render the component in a test DOM and assert the presence of specific text fields (name, email). | The layout utilizes CSS flexbox or grid to adapt to mobile screen sizes. |